

Data Analysis for

“On the Increasing Partisan Divide of the U.S. Supreme Court”

Table of contents

Setup	4
Version	4
Libraries	4
Library Versions	5
Analysis Parameters	7
Purpose	7
Created objects	7
Project Directories and Input Files	8
Purpose	8
Created objects	8
Case Salience Index	10
Definition	10
Purpose	11
Created objects	11
Supreme Court Database	16
Purpose	16
Created objects	17
Appointing-President Party and Justice Service Coding	18
Purpose	18
Created objects	18
Construction of Appointing-Party Alignment	23
Absolute Phi Coefficient	23
Margin-Normalized Phi	24
Case-Level Voting Coalitions	27
Purpose	27
Created objects	27
Merge of Salience and Voting Data	32
Purpose	32
Created objects	32
Merge and Coding Diagnostics	35
Purpose	35
Created objects	35

Descriptive Analysis	40
Descriptive Analysis Setup	40
Purpose	40
Created objects	40
Historical Strict Appointing-Party Splits	42
Purpose	42
Created objects	42
Natural Court Composition and Historical Periods	45
Purpose	45
Created objects	45
Appointing-Party Alignment by Natural Court	49
Purpose	49
Created objects	49
Vote-Division Proportions by Case Salience	53
Purpose	53
Created objects	53
Five-Term Rolling Appointing-Party Alignment Across CSI Thresholds	55
Purpose	55
Created objects	56
High- Versus Lower-Salience Cases Across CSI Thresholds	67
Purpose	67
Created objects	67
Five-Term Rolling Salience-Band Gaps	68
Purpose	68
Created objects	68
Pre-/Post-2010 Descriptive Comparison	79
Purpose	79
Created objects	79
Estimated Breakpoint in the Salience–Alignment Relationship	84
Purpose	84
Created objects	85
Salience Gap by Natural Court	95
Purpose	95
Created objects	95
Strict Appointing-Party Split Proportions	104
Purpose	104
Created objects	104
Predictive Analysis of the Salience–Alignment Relationship	112
Vote Division by Case Salience	112
Purpose	112

Created objects	112
CSI Regression Model	115
Purpose	115
Created objects	115
Breakpoint Diagnostics	121
Purpose	121
Created objects	121
Main Regression Models	132
Purpose	132
Created objects	132
Inferential Robustness Checks	139
Purpose	139
Created objects	139

Setup

Version

```
# R Version
R.version
```

```
platform      -
arch           x86_64
os             mingw32
crt            ucrt
system        x86_64, mingw32
status
major          4
minor          4.1
year           2024
month          06
day            14
svn rev        86737
language       R
version.string R version 4.4.1 (2024-06-14 ucrt)
nickname       Race for Your Life
```

Libraries

```
# ---- package setup -----
required_pkgs <- c(
  "dplyr", "ggplot2", "readxl", "stringr", "purrr", "readr", "tidyr",
  "tibble", "knitr", "kableExtra", "sandwich", "lmtest", "scales", "devEMF"
)

install_if_missing <- function(pkgs) {
  missing <- pkgs[!pkgs %in% rownames(installed.packages())]

  if (length(missing) > 0) {
    message(
      "Installing missing packages: ", paste(missing, collapse = ", ")
    )
  }
}
```

```

    install.packages(missing, dependencies = TRUE)
  }
}

load_quietly <- function(pkgs) {
  for (pkg in pkgs) {
    suppressPackageStartupMessages(
      suppressWarnings(library(pkg, character.only = TRUE)))
  }
}

install_if_missing(required_pkgs)
load_quietly(required_pkgs)

```

Library Versions

```

# Package versions
for (pkg in required_pkgs) {

  ver <- if (requireNamespace(pkg, quietly = TRUE)) {
    as.character(packageVersion(pkg))
  } else {
    "NOT INSTALLED"
  }

  cat(pkg, ":", ver, "\n")
}

```

```

dplyr : 1.2.0
ggplot2 : 4.0.2
readxl : 1.4.5
stringr : 1.6.0
purrr : 1.2.1
readr : 2.1.5
tidyr : 1.3.1
tibble : 3.2.1
knitr : 1.49
kableExtra : 1.4.0
sandwich : 3.1.1
lmtest : 0.9.40

```

scales : 1.4.0
devEMF : 4.5.1

Analysis Parameters

Purpose

Define the analysis period, the CSI thresholds used in descriptive analyses, and the width of the rolling time window.

The analysis covers **October Terms 1953–2024**. Because CSI is a continuous measure and does not define an official threshold for “high salience,” the descriptive analyses will be repeated across several increasingly restrictive thresholds: **CSI 3, 4, 5, 6, and 7**. Later inferential analyses can additionally use CSI as a continuous variable.

Created objects

- **analysis_start_term** — first Supreme Court term included (1953).
- **analysis_end_term** — final Supreme Court term included (2024).
- **csi_cutoffs** — vector of CSI thresholds used in descriptive analyses (3:7).
- **rolling_window_terms** — number of Supreme Court terms pooled in the rolling-window analyses (5).

```
# --- Analysis parameters -----  
  
analysis_start_term <- 1953L  
analysis_end_term   <- 2024L  
  
# CSI has no official "high salience" cutoff.  
# Descriptive analyses are therefore repeated across several thresholds.  
csi_cutoffs <- 3:7  
  
# Main rolling-window width used for descriptive time-series analyses.  
rolling_window_terms <- 5L  
  
message(  
  "Analysis period: OT", analysis_start_term, "-OT", analysis_end_term)
```

Analysis period: OT1953-OT2024

```
message(  
  "CSI cutoffs: ", paste(paste0("CSI >= ", csi_cutoffs), collapse = ", ")  
)
```

CSI cutoffs: CSI >= 3, CSI >= 4, CSI >= 5, CSI >= 6, CSI >= 7

```
message(  
  "Rolling-window width: ", rolling_window_terms,  
  " Supreme Court terms (approximately ", rolling_window_terms, " years)")
```

Rolling-window width: 5 Supreme Court terms (approximately 5 years)

Project Directories and Input Files

Purpose

Define the project-level folders and locate the three source files required for the analysis. The code checks the conventional `data/` directory first and then the current working directory. This makes the Quarto document portable while preserving the same project structure used in the other analysis files.

The substantive analysis uses:

- the original **Case Salience Index (CSI)** covering 1953–2014,
- the updated **CSI** covering 2015–2024,
- the **Supreme Court Database (SCDB) Justice-Centered Citation-Organized dataset**.

The main analysis therefore covers **October Terms 1953–2024**. OT2025/2025–26 is not appended to the principal time series because a directly comparable CSI measure is not yet available for that term.

Created objects

- **data_dir** — preferred directory containing the raw source files.
- **fig_dir** — output directory for figures.
- **table_dir** — output directory for CSV tables.
- **analysis_dir** — output directory for derived case-level datasets.
- **resolve_input_path()** — helper function locating a file in `data/` or the working directory.
- **old_csi_file** — path to `CSI-1953-2014.xls`.

- **new_csi_zip** — path to CSI_2015-2024.zip.
- **scdb_zip** — path to SCDB_2025_01_justiceCentered_Citation.csv_.zip.

```
# --- Project directories + source-file discovery -----

data_dir      <- "data"
fig_dir       <- "figures"
table_dir     <- "tables"
analysis_dir  <- "analysis_data"

for (dir_i in c(
  data_dir, fig_dir, table_dir, analysis_dir)) {
  if (!dir.exists(dir_i)) {
    dir.create(dir_i, recursive = TRUE)
  }
}

resolve_input_path <- function(filename) {

  candidate_data <- file.path(data_dir, filename)

  candidate_root <- filename

  if (file.exists(candidate_data)) {
    return(candidate_data)
  }

  if (file.exists(candidate_root)) {
    return(candidate_root)
  }

  stop(
    "Required input file not found: ", filename, "\nChecked:\n", " - ",
    candidate_data, "\n - ", candidate_root)
}

old_csi_file <- resolve_input_path("CSI-1953-2014.xls")

new_csi_zip <- resolve_input_path("CSI_2015-2024.zip")

scdb_zip <- resolve_input_path(
  "SCDB_2025_01_justiceCentered_Citation.csv_.zip")
```

```
message("CSI 1953-2014: ", old_csi_file)
```

CSI 1953-2014: data/CSI-1953-2014.xls

```
message("CSI 2015-2024: ", new_csi_zip)
```

CSI 2015-2024: data/CSI_2015-2024.zip

```
message("SCDB:          ", scdb_zip)
```

SCDB: data/SCDB_2025_01_justiceCentered_Citation.csv_.zip

Case Salience Index

Definition

The **Case Salience Index (CSI)** is a continuous measure of contemporaneous media attention to each orally argued Supreme Court case.

For each decision, four newspapers are scored separately:

- *Los Angeles Times*
- *Chicago Tribune*
- *Washington Post*
- *New York Times*

Each newspaper contributes:

- **0** — no coverage,
- **1** — coverage, but not on the front page,
- **2** — front-page coverage.

The four newspaper scores are then added:

$$CSI = LA + Chicago + Washington + NYT$$

so that:

$$0 \leq CSI \leq 8.$$

Higher CSI values therefore indicate greater contemporaneous public/media salience. Most importantly, the CSI is a **salience measure**, not a direct measure of ideological content or partisan controversy.

Because the CSI does not provide an official threshold defining a “high-salience” case, the descriptive analyses examine several thresholds: **CSI 3**, **CSI 4**, **CSI 5**, **CSI 6**, and **CSI 7**. This allows the analysis to assess whether the historical results depend on the chosen salience threshold. Inferential analyses will additionally retain CSI as a continuous variable.

Purpose

The following chunk reads the original and updated CSI files, harmonizes their variable names, calculates the total CSI score where necessary, combines both periods, and constructs a continuous case-salience dataframe covering **OT1953–OT2024**.

The resulting dataset then preserves the continuous CSI score as well as the four newspaper-specific component scores. No single high-salience threshold is imposed at the data-preparation stage.

Created objects

- **extract_one_csv()** — helper function that extracts the relevant CSV from a ZIP archive.
- **canonical_name()** — helper function that standardizes column names for matching.
- **pick_column()** — defensive helper selecting a source column from a set of plausible names.
- **normalise_csi()** — converts either CSI source file into a common structure.
- **csi_old_raw** — raw 1953–2014 CSI import.
- **csi_new_raw** — raw 2015–2024 CSI import.
- **csi_old**, **csi_new** — normalized CSI datasets.

- **CSI_CaseUniverse** — combined CSI case universe for OT1953–OT2024.
- **CSI_threshold_counts** — diagnostic table reporting the number of cases meeting each CSI threshold defined in `csi_cutoffs`.

```
# --- CSI: read, harmonize, and combine -----

extract_one_csv <- function(zipfile, preferred_pattern = NULL) {

  members <- utils::unzip(zipfile, list = TRUE)$Name

  members <- members[
    grepl("\\.csv$", members, ignore.case = TRUE) &
    !grepl("__MACOSX", members)]

  if (!is.null(preferred_pattern)) {

    preferred <- members[
      grepl(preferred_pattern, members, ignore.case = TRUE)]

    if (length(preferred) > 0) {
      members <- preferred
    }
  }

  if (length(members) == 0) {
    stop("No CSV file found inside: ", zipfile)
  }

  exdir <- tempfile("unz_")
  dir.create(exdir)

  utils::unzip(zipfile, files = members[1], exdir = exdir)

  file.path(exdir, members[1])
}

canonical_name <- function(x) {

  tolower(gsub("[^a-z0-9]", "", x))
}

pick_column <- function(df, candidates, required = TRUE) {
```

```

nms      <- canonical_name(names(df))
candidates <- canonical_name(candidates)

hit <- which(nms %in% candidates)

if (length(hit) == 0) {

  if (required) {

    stop(
      "Could not find a required CSI column.\n", "Candidates: ",
      paste(candidates, collapse = ", "), "\nAvailable columns:\n",
      paste(names(df), collapse = ", "))

  } else {

    return(NULL)
  }
}

df[[hit[1]]]
}

normalise_csi <- function(df, source_label) {

  case_id <- pick_column(df, c("caseId", "case_id"))

  la <- pick_column(
    df, c("laScore", "la_score", "losangelesscore", "latimescore"))

  ch <- pick_column(
    df, c(
      "chScore", "chiScore", "ch_score", "chicagoscore",
      "chicagotribunescore"))

  wash <- pick_column(
    df, c(
      "washScore", "waScore", "wash_score", "washingtonscore",
      "washingtonpostscore"))

  ny <- pick_column(
    df, c("nyScore", "ny_score", "nytimescore", "newyorktimescore"))

```

```

supplied_csi <- pick_column(
  df, c("csi", "caseSalienceIndex"), required = FALSE)

out <- tibble::tibble(
  caseId    = as.character(case_id), laScore    = as.numeric(la),
  chScore   = as.numeric(ch), washScore = as.numeric(wash),
  nyScore   = as.numeric(ny))

if (is.null(supplied_csi)) {

  out <- out %>%
    dplyr::mutate(csi = laScore + chScore + washScore + nyScore)

} else {

  out <- out %>% dplyr::mutate(csi = as.numeric(supplied_csi))
}

out %>% dplyr::mutate(
  term_csi   = as.integer(substr(caseId, 1, 4)),
  csi_source = source_label)
}

# -----
# Original CSI: 1953-2014
# -----

csi_old_raw <- readxl::read_excel(old_csi_file, sheet = 1)

csi_old <- normalise_csi(csi_old_raw, "Collins & Cooper 1953-2014")

# -----
# Updated CSI: 2015-2024
# -----

new_csi_csv <- extract_one_csv(
  new_csi_zip, preferred_pattern = "CSI_2015-2024")

csi_new_raw <- readr::read_csv(new_csi_csv, show_col_types = FALSE)

csi_new <- normalise_csi(csi_new_raw, "Updated CSI 2015-2024")

```

```

# -----
# Combined CSI case universe
# -----

CSI_CaseUniverse <- dplyr::bind_rows(csi_old, csi_new) %>% dplyr::filter(
  term_csi >= analysis_start_term, term_csi <= analysis_end_term) %>%
dplyr::distinct(caseId, .keep_all = TRUE) %>%
dplyr::arrange(term_csi, caseId)

# -----
# Threshold diagnostics
# -----

CSI_threshold_counts <- purrr::map_dfr(
  csi_cutoffs, function(k) {

    tibble::tibble(
      CSI_Cutoff = k,
      N_cases = sum(CSI_CaseUniverse$csi >= k, na.rm = TRUE),
      Share_cases = mean(CSI_CaseUniverse$csi >= k, na.rm = TRUE))
  })

# -----
# Console diagnostics
# -----

message(
  "CSI case universe created: rows = ", nrow(CSI_CaseUniverse),
  " | OT range = ", min(CSI_CaseUniverse$term_csi, na.rm = TRUE), "-",
  max(CSI_CaseUniverse$term_csi, na.rm = TRUE))

```

```
CSI case universe created: rows = 7604 | OT range = 1953-2024
```

```
cat("\nCSI threshold counts:\n")
```

```
CSI threshold counts:
```

```
print(CSI_threshold_counts)
```

```
# A tibble: 5 x 3
  CSI_Cutoff N_cases Share_cases
    <int>    <int>      <dbl>
1       3     3458      0.455
2       4     2449      0.322
3       5     1497      0.197
4       6      950      0.125
5       7      505      0.0664
```

```
# -----
# Quality assurance
# -----

stopifnot(
  min(CSI_CaseUniverse$term_csi, na.rm = TRUE) == analysis_start_term)

stopifnot(
  max(CSI_CaseUniverse$term_csi, na.rm = TRUE) == analysis_end_term)

stopifnot(
  all(CSI_CaseUniverse$csi >= 0 & CSI_CaseUniverse$csi <= 8, na.rm = TRUE)
)

stopifnot(!anyDuplicated(CSI_CaseUniverse$caseId))
```

Supreme Court Database

Purpose

The following code reads the justice-centered Supreme Court Database (SCDB) data and restricts the file to the principal analysis period, **OT1953–OT2024**.

Justice-centered data are required because the dependent variable will be reconstructed from the composition of the majority and dissenting coalitions within each individual case. The justice-centered file provides one observation for each justice participating in each case.

The SCDB `majority` field will later be used to identify each justice's disposition side:

- `majority == 2` — majority/disposition side,
- `majority == 1` — dissenting side.

The SCDB **naturalCourt** variable is retained because later descriptive analyses will summarize appointing-party alignment across periods of unchanged Supreme Court membership rather than relying only on arbitrary calendar decades.

Created objects

- **scdb_csv** — path to the extracted justice-centered SCDB CSV file.
- **SCDB_JusticeVotes_raw** — justice-centered SCDB observations for OT1953–OT2024, retaining one row per justice × case and all source variables needed for subsequent coalition and Natural Court analyses.

```
# --- SCDB: read justice-centered citation-organized dataset -----  
  
scdb_csv <- extract_one_csv(  
  scdb_zip, preferred_pattern = "justiceCentered_Citation")  
  
SCDB_JusticeVotes_raw <- readr::read_csv(  
  scdb_csv, show_col_types = FALSE) %>%  
  dplyr::filter(term >= analysis_start_term, term <= analysis_end_term)  
  
message(  
  "SCDB justice-vote rows = ", nrow(SCDB_JusticeVotes_raw), " | cases = ",  
  dplyr::n_distinct(SCDB_JusticeVotes_raw$caseId), " | OT range = ",  
  min(SCDB_JusticeVotes_raw$term, na.rm = TRUE), "-",  
  max(SCDB_JusticeVotes_raw$term, na.rm = TRUE))
```

```
SCDB justice-vote rows = 76337 | cases = 8529 | OT range = 1953-2024
```

```
message(  
  "Natural Courts represented = ",  
  dplyr::n_distinct(SCDB_JusticeVotes_raw$naturalCourt, na.rm = TRUE))
```

```
Natural Courts represented = 34
```

```
# -----  
# Quality assurance  
# -----  
  
stopifnot(  
  min(SCDB_JusticeVotes_raw$term, na.rm = TRUE) == analysis_start_term)
```

```

stopifnot(
  max(SCDB_JusticeVotes_raw$term, na.rm = TRUE) == analysis_end_term)

stopifnot("naturalCourt" %in% names(SCDB_JusticeVotes_raw))

stopifnot("majority" %in% names(SCDB_JusticeVotes_raw))

stopifnot("justice" %in% names(SCDB_JusticeVotes_raw))

stopifnot("caseId" %in% names(SCDB_JusticeVotes_raw))

```

Appointing-President Party and Justice Service Coding

Purpose

The following chunk create the justice-level metadata required to measure **appointing-party alignment** and to identify changes in Supreme Court membership over the full OT1953–OT2024 analysis period.

The principal partisan variable is the party of the **president who originally appointed the justice to the Supreme Court**, rather than an ex-post ideological classification of the justice. Historical crossover justices therefore retain the partisan origin of their appointment.

The metadata additionally record each justice’s **service-start date** and **service-end date**. Service start is defined using the date on which the justice took the judicial oath and began service on the Court.

Court-composition periods themselves are not inferred solely from these dates. Subsequent analyses use the SCDB **naturalCourt** variable, which identifies periods of unchanged Court personnel. The justice service dates are retained so that Natural Courts can later be described in terms of which justice entered or departed and when the personnel transition occurred.

Created objects

- **justice_metadata** — justice-level crosswalk containing appointing president, appointing-president party, service-start date, and service-end date.
- **justice_transitions** — chronologically ordered table showing changes in Supreme Court membership during the analysis period.
- **unmapped_justices** — diagnostic table identifying any SCDB justice without a corresponding metadata record.

- **SCDB_JusticeVotes** — justice-centered SCDB data with justice metadata and provisional majority/dissent-side coding attached.

```
# --- Appointing-president party + justice service metadata -----
#
# Definitions:
#   R = appointed to the Supreme Court by a Republican president
#   D = appointed to the Supreme Court by a Democratic president
#
# service_start:
#   Date on which the justice took the judicial oath and began service.
#
# service_end:
#   Date Supreme Court service terminated.
#   NA indicates that the justice remained on the Court through OT2024.
#
# Important:
#   Appointing party is NOT an ideological coding of the justice.
#   It records the party of the president who originally appointed the
#   individual to the Supreme Court.

justice_metadata <- tibble::tribble(
  ~justiceName, ~appointing_president, ~appointing_party,
  ~service_start, ~service_end,

  "HLBlack", "Franklin D. Roosevelt", "D", "1937-08-19", "1971-09-17",
  "SFReed", "Franklin D. Roosevelt", "D", "1938-01-31", "1957-02-25",
  "FFrankfurter", "Franklin D. Roosevelt", "D", "1939-01-30", "1962-08-28",
  "WODouglas", "Franklin D. Roosevelt", "D", "1939-04-17", "1975-11-12",
  "RHJackson", "Franklin D. Roosevelt", "D", "1941-07-11", "1954-10-09",

  "HHBurton", "Harry S. Truman", "D", "1945-10-01", "1958-10-13",
  "TCClark", "Harry S. Truman", "D", "1949-08-24", "1967-06-12",
  "SMinton", "Harry S. Truman", "D", "1949-10-12", "1956-10-15",

  "EWarren", "Dwight D. Eisenhower", "R", "1953-10-05", "1969-06-23",
  "JHarlan2", "Dwight D. Eisenhower", "R", "1955-03-28", "1971-09-23",
  "WJBrennan", "Dwight D. Eisenhower", "R", "1956-10-16", "1990-07-20",
  "CEWhittaker", "Dwight D. Eisenhower", "R", "1957-03-25", "1962-03-31",
  "PStewart", "Dwight D. Eisenhower", "R", "1958-10-14", "1981-07-03",

  "BRWhite", "John F. Kennedy", "D", "1962-04-16", "1993-06-28",
  "AJGoldberg", "John F. Kennedy", "D", "1962-10-01", "1965-07-25",
```

```

"AFortas", "Lyndon B. Johnson", "D", "1965-10-04", "1969-05-14",
"TMarshall", "Lyndon B. Johnson", "D", "1967-10-02", "1991-10-01",

"WEBurger", "Richard Nixon", "R", "1969-06-23", "1986-09-26",
"HABlackmun", "Richard Nixon", "R", "1970-06-09", "1994-08-03",
"LFPowell", "Richard Nixon", "R", "1972-01-07", "1987-06-26",
"WHRehnquist", "Richard Nixon", "R", "1972-01-07", "2005-09-03",

"JPStevens", "Gerald Ford", "R", "1975-12-19", "2010-06-29",

"SDOConnor", "Ronald Reagan", "R", "1981-09-25", "2006-01-31",
"AScalia", "Ronald Reagan", "R", "1986-09-26", "2016-02-13",
"AMKennedy", "Ronald Reagan", "R", "1988-02-18", "2018-07-31",

"DHSouter", "George H. W. Bush", "R", "1990-10-09", "2009-06-29",
"CThomas", "George H. W. Bush", "R", "1991-10-23", NA_character_,

"RBGinsburg", "Bill Clinton", "D", "1993-08-10", "2020-09-18",
"SGBreyer", "Bill Clinton", "D", "1994-08-03", "2022-06-30",

"JGRoberts", "George W. Bush", "R", "2005-09-29", NA_character_,
"SAAlito", "George W. Bush", "R", "2006-01-31", NA_character_,

"SSotomayor", "Barack Obama", "D", "2009-08-08", NA_character_,
"EKagan", "Barack Obama", "D", "2010-08-07", NA_character_,

"NMGorsuch", "Donald Trump", "R", "2017-04-10", NA_character_,
"BMKavanaugh", "Donald Trump", "R", "2018-10-06", NA_character_,
"ACBarrett", "Donald Trump", "R", "2020-10-27", NA_character_,

"KBJackson", "Joe Biden", "D", "2022-06-30", NA_character_
) %>% dplyr::mutate(
  service_start = as.Date(service_start),
  service_end   = as.Date(service_end))

# -----
# Membership transitions during the analysis period
# -----
#
# This object provides a chronological record of justices entering the Court.
# It will later help label changes between Natural Courts.
#

```

```

# Justices who entered before OT1953 remain in justice_metadata but are not
# treated as new appointments during the analysis period.

justice_transitions <- justice_metadata %>% dplyr::filter(
  service_start >= as.Date("1953-01-01"),
  service_start <= as.Date("2024-12-31")) %>%
dplyr::arrange(service_start) %>% dplyr::select(
  justiceName, service_start, service_end, appointing_president,
  appointing_party)

# -----
# QA: every justice appearing in the analysis period must be mapped
# -----

unmapped_justices <- SCDB_JusticeVotes_raw %>%
  dplyr::distinct(justiceName) %>%
  dplyr::anti_join(justice_metadata, by = "justiceName")

if (nrow(unmapped_justices) > 0) {

  print(unmapped_justices)

  stop(
    "At least one justice in the SCDB analysis period lacks ",
    "justice-level metadata.")
}

# -----
# Attach justice metadata + provisional majority/dissent side
# -----
#
# No observations are dropped here.
# Rows that cannot be classified as majority == 1 or majority == 2 remain
# in the dataset with side = NA and will be handled explicitly when the
# analytic voting sample is constructed.

SCDB_JusticeVotes <- SCDB_JusticeVotes_raw %>%
  dplyr::left_join(justice_metadata, by = "justiceName") %>%
  dplyr::mutate(
    side = dplyr::case_when(
      majority == 2 ~ "majority", majority == 1 ~ "dissent",
      TRUE ~ NA_character_))

```

```
# -----
# Console diagnostics
# -----
```

```
message("Justice metadata records = ", nrow(justice_metadata))
```

Justice metadata records = 37

```
message(
  "Justice entries during analysis period = ", nrow(justice_transitions))
```

Justice entries during analysis period = 29

```
message(
  "Natural Courts in SCDB analysis period = ",
  dplyr::n_distinct(SCDB_JusticeVotes$naturalCourt, na.rm = TRUE))
```

Natural Courts in SCDB analysis period = 34

```
message(
  "Justice vote coding complete: ", sum(!is.na(SCDB_JusticeVotes$side)),
  " rows have observable majority/dissent sides.")
```

Justice vote coding complete: 73851 rows have observable majority/dissent sides.

```
# -----
# Quality assurance
# -----
```

```
stopifnot(nrow(unmapped_justices) == 0)

stopifnot(all(justice_metadata$appointing_party %in% c("D", "R")))

stopifnot(!anyDuplicated(justice_metadata$justiceName))

stopifnot(
  all(
    is.na(justice_metadata$service_end) |
    justice_metadata$service_end >= justice_metadata$service_start))
```

Construction of Appointing-Party Alignment

Absolute Phi Coefficient

Purpose

Measure the **strength of association between appointing-president party and disposition side** separately for each Supreme Court case for which both dimensions are observable.

For each case, participating justice votes form the following 2×2 table:

	Majority	Dissent
Republican-appointed	(a)	(b)
Democratic-appointed	(c)	(d)

The phi coefficient is:

$$\phi = \frac{ad - bc}{\sqrt{(a+b)(c+d)(b+d)}}$$

Because the substantive question concerns the **strength of party sorting**, rather than whether Republican- or Democratic-appointed justices occupy the majority side, the analysis uses the absolute value:

$$|\phi|$$

For any coefficient $|\phi|$, the interpretation is:

- ($|\phi| = 1$) — perfect appointing-party sorting.
- ($|\phi| \approx 0$) — appointing party has little association with the side taken by a justice.
- Intermediate values represent partial party sorting.

Created objects

- `phi_abs_from_counts()` — helper computing absolute phi from the four cell counts.

```

# --- Absolute phi coefficient -----
#
# Input:
#
#           Majority    Dissent
# R-appointed      a      b
# D-appointed      c      d
#
# Output:
#   Absolute phi, ranging from 0 to 1 where calculable.
#
#
# The coefficient is returned as NA when it is mathematically undefined
# because one of the row or column dimensions has no variation.

phi_abs_from_counts <- function(a, b, c, d) {

  denominator <- sqrt((a + b) * (c + d) * (a + c) * (b + d))

  ifelse(denominator > 0, abs((a * d - b * c) / denominator), NA_real_)
}

```

Margin-Normalized Phi

Purpose

The following chunk calculates the **maximum absolute phi coefficient that is mathematically attainable** given the observed appointing-party composition and majority/dissent vote margins in each case, and express the observed absolute phi relative to that maximum.

This adjustment is useful because raw $|\phi|$ can be constrained by the marginal totals of the $2 \times 2 \times 2$ table. For example, a Court with a highly unequal number of Republican- and Democratic-appointed justices may not be able to attain $|\phi| = 1$ for every possible vote split.

The margin-normalized measure is therefore:

$$|\phi|_{\text{normalized}} = \frac{|\phi|_{\text{observed}}}{|\phi|_{\text{maximum feasible}}}$$

It is interpreted similarly to $|\phi|$:

- **0** — no observed appointing-party association.

- **1** — the strongest appointing-party association mathematically possible given the observed margins.
- Intermediate values indicate the proportion of the maximum feasible association that is realized.

The raw absolute phi coefficient remains the **principal outcome measure**. Margin-normalized phi is retained as a robustness measure for comparisons across Court compositions and vote margins.

Created objects

- **phi_max_from_margins()** — calculates the maximum absolute phi attainable given the observed row and column margins.
- **phi_normalized_from_counts()** — calculates observed absolute phi as a proportion of the maximum attainable absolute phi.

```
# --- Margin-normalized absolute phi -----

phi_max_from_margins <- function(a, b, c, d) {

  r_total <- a + b
  d_total <- c + d

  majority_total <- a + c
  dissent_total <- b + d

  n_total <- r_total + d_total

  denominator <- sqrt(r_total * d_total * majority_total * dissent_total)

  a_min <- pmax(0, r_total + majority_total - n_total)

  a_max <- pmin(r_total, majority_total)

  numerator_min <- abs(a_min * n_total - r_total * majority_total)

  numerator_max <- abs(a_max * n_total - r_total * majority_total)

  maximum_numerator <- pmax(numerator_min, numerator_max)
```

```

dplyr::if_else(
  denominator > 0, maximum_numerator / denominator, NA_real_)
}

phi_normalized_from_counts <- function(a, b, c, d) {

  observed_phi <- phi_abs_from_counts(a = a, b = b, c = c, d = d)

  maximum_phi <- phi_max_from_margins(a = a, b = b, c = c, d = d)

  dplyr::if_else(
    !is.na(maximum_phi) & maximum_phi > 0, observed_phi / maximum_phi,
    NA_real_)
}

```

Case-Level Voting Coalitions

Purpose

The following chunk collapses the justice-centered SCDB file to one row per Supreme Court case. For each case, count Republican- and Democratic-appointed justices on the majority and dissenting sides, calculate the principal absolute phi measure and the margin-normalized robustness measure, and identify strict appointing-party splits.

A **strict appointing-party split** is coded TRUE when:

- all participating Republican-appointed justices are on one disposition side;
- all participating Democratic-appointed justices are on the opposite disposition side;
- both appointing-party groups are represented; and
- the case is non-unanimous.

Cases decided by an equal division are not assigned a majority/dissent party-alignment score because the SCDB majority/minority coding does not provide a suitable majority-versus-dissent distinction for the present analysis.

The resulting case-level data also retain the SCDB **naturalCourt** identifier so that subsequent analyses can compare appointing-party alignment across periods of unchanged Court membership.

Created objects

- **SCDB_CaseVotes** — one row per SCDB case containing case metadata, Natural Court identifier, vote counts, appointing-party composition, party-by-side cell counts, raw absolute phi, margin-normalized phi, and strict appointing-party split status.

```
# --- Collapse justice votes to one row per case -----  
#  
# Purpose:  
#   Reconstruct the 2x2 appointing-party × disposition-side table for every  
#   Supreme Court case and calculate the principal and robustness measures  
#   of appointing-party alignment.  
#  
# Key output columns:  
#   R_majority  
#   R_dissent  
#   D_majority  
#   D_dissent
```

```

# R_total
# D_total
# phi_abs
# phi_max
# phi_normalized
# strict_party_split
# naturalCourt

SCDB_CaseVotes <- SCDB_JusticeVotes %>% dplyr::group_by(caseId) %>%
  dplyr::summarise(

    # -----
    # Case metadata
    # -----

    term_scdb      = dplyr::first(term),
    dateDecision   = dplyr::first(dateDecision),
    caseName       = dplyr::first(caseName),
    usCite         = dplyr::first(usCite),
    docket         = dplyr::first(docket),
    issueArea      = dplyr::first(issueArea),
    naturalCourt   = dplyr::first(naturalCourt),

    majVotes = dplyr::first(majVotes), minVotes = dplyr::first(minVotes),

    # -----
    # Appointing-party × disposition-side cell counts
    # -----

    R_majority = sum(
      appointing_party == "R" & side == "majority", na.rm = TRUE),

    R_dissent = sum(
      appointing_party == "R" & side == "dissent", na.rm = TRUE),

    D_majority = sum(
      appointing_party == "D" & side == "majority", na.rm = TRUE),

    D_dissent = sum(
      appointing_party == "D" & side == "dissent", na.rm = TRUE),

    n_observed_sides = sum(!is.na(side)),

```

```

.groups = "drop") %>% dplyr::mutate(

# -----
# Court composition among participating justices
# -----

R_total = R_majority + R_dissent,

D_total = D_majority + D_dissent,

n_party_coded = R_total + D_total,

# -----
# Equal divisions and observability
# -----

equal_division = !is.na(majVotes) & !is.na(minVotes) &
  majVotes == minVotes,

observable_sides = !is.na(majVotes) & !is.na(minVotes) &
  !equal_division & n_observed_sides == (majVotes + minVotes),

has_both_appointment_parties = R_total > 0 & D_total > 0,

# -----
# Principal absolute-phi outcome
# -----

phi_abs = dplyr::if_else(
  observable_sides & has_both_appointment_parties,

  phi_abs_from_counts(R_majority, R_dissent, D_majority, D_dissent),

  NA_real_),

# -----
# Maximum feasible phi under observed margins
# -----

phi_max = dplyr::if_else(
  observable_sides & has_both_appointment_parties,

```

```

phi_max_from_margins(R_majority, R_dissent, D_majority, D_dissent),

NA_real_),

# -----
# Margin-normalized phi
# -----

phi_normalized = dplyr::if_else(
  observable_sides & has_both_appointment_parties,

  phi_normalized_from_counts(
    R_majority, R_dissent, D_majority, D_dissent),

  NA_real_),

# -----
# Strict appointing-party split
# -----

strict_party_split = dplyr::case_when(

  # Unanimous cases are not strict party splits.
  !is.na(minVotes) & minVotes == 0 ~ FALSE,

  # Cases without observable majority/dissent sides cannot be classified.
  !observable_sides ~ NA,

  # Both appointing-party groups must be represented.
  !has_both_appointment_parties ~ NA,

  # All R-appointed justices majority;
  # all D-appointed justices dissent.
  R_majority > 0 & R_dissent == 0 & D_majority == 0 &
    D_dissent > 0 ~ TRUE,

  # All R-appointed justices dissent;
  # all D-appointed justices majority.
  R_majority == 0 & R_dissent > 0 & D_majority > 0 &
    D_dissent == 0 ~ TRUE,

  TRUE ~ FALSE))

```

```
# -----
# Console diagnostics
# -----

message(
  "Case-level voting table created: ", nrow(SCDB_CaseVotes), " SCDB cases."
)
```

Case-level voting table created: 8529 SCDB cases.

```
message(
  "Cases with calculable |phi|: ", sum(!is.na(SCDB_CaseVotes$phi_abs)))
```

Cases with calculable |phi|: 5103

```
message(
  "Cases with calculable margin-normalized |phi|: ",
  sum(!is.na(SCDB_CaseVotes$phi_normalized)))
```

Cases with calculable margin-normalized |phi|: 5103

```
message(
  "Natural Courts represented: ",
  dplyr::n_distinct(SCDB_CaseVotes$naturalCourt, na.rm = TRUE))
```

Natural Courts represented: 34

Merge of Saliency and Voting Data

Purpose

Merge the independent Case Saliency Index data with the reconstructed case-level voting-coalition data.

The principal analysis sample consists of:

1. a case contained in the CSI orally argued case universe;
2. a case between **OT1953 and OT2024**;
3. at least **two minority votes**; and
4. a calculable absolute phi coefficient.

The requirement of at least two minority votes defines the principal sample of **contested decisions**. This focuses the main analysis on cases exhibiting more than an isolated dissent. Alternative definitions using all non-unanimous cases and more closely divided cases are examined as robustness checks.

No single CSI cutoff is imposed at this stage. CSI remains continuous, and subsequent descriptive analyses evaluate the thresholds defined globally in `csi_cutoffs` (3:7).

Created objects

- **SCOTUS_CaseData** — merged case-level CSI and SCDB dataset for OT1953–OT2024.
- **SCOTUS_Contested** — principal sample containing cases with at least two minority votes and calculable absolute phi.
- **SCOTUS_NonUnanimous** — robustness sample containing all cases with at least one minority vote and calculable absolute phi.
- **SCOTUS_CloselyDivided** — robustness sample containing cases with at least three minority votes and calculable absolute phi.

```
# --- Merge saliency and voting-coalition data -----  
#  
# Main sample:  
#   - CSI orally argued case universe  
#   - principal analysis period  
#   - >= 2 minority votes  
#   - calculable absolute phi  
#
```

```

# Robustness samples:
#   - all non-unanimous cases
#   - >= 3 minority votes
#
# CSI remains continuous here. Threshold-specific analyses are generated
# later using the globally defined csi_cutoffs vector.

SCOTUS_CaseData <- CSI_CaseUniverse %>%
  dplyr::left_join(SCDB_CaseVotes, by = "caseId") %>% dplyr::mutate(

    term = dplyr::coalesce(term_scdb, term_csi),

    contested_2plus = !is.na(minVotes) & minVotes >= 2,

    closely_divided = !is.na(minVotes) & minVotes >= 3) %>% dplyr::filter(
  term >= analysis_start_term, term <= analysis_end_term) %>%
  dplyr::arrange(term, caseId)

# -----
# Principal contested sample
# -----

SCOTUS_Contested <- SCOTUS_CaseData %>%
  dplyr::filter(contested_2plus, !is.na(phi_abs))

# -----
# Robustness sample: all non-unanimous cases
# -----

SCOTUS_NonUnanimous <- SCOTUS_CaseData %>%
  dplyr::filter(!is.na(minVotes), minVotes >= 1, !is.na(phi_abs))

# -----
# Robustness sample: more closely divided cases
# -----

SCOTUS_CloselyDivided <- SCOTUS_CaseData %>%
  dplyr::filter(closely_divided, !is.na(phi_abs))

# -----
# Console diagnostics
# -----

```

```
message("Merged CSI universe: ", nrow(SCOTUS_CaseData), " cases.")
```

Merged CSI universe: 7604 cases.

```
message(
  "Merged analysis period: OT", min(SCOTUS_CaseData$term, na.rm = TRUE),
  "-OT", max(SCOTUS_CaseData$term, na.rm = TRUE))
```

Merged analysis period: OT1953-OT2024

```
message(
  "Main contested sample (2+ minority votes): ", nrow(SCOTUS_Contested),
  " cases.")
```

Main contested sample (2+ minority votes): 3753 cases.

```
message(
  "Main contested cases with normalized phi: ",
  sum(!is.na(SCOTUS_Contested$phi_normalized)), " cases.")
```

Main contested cases with normalized phi: 3753 cases.

Merge and Coding Diagnostics

Purpose

Verify the integrity of the merged case-level dataset before undertaking substantive analyses.

The diagnostics assess:

- temporal coverage of the CSI–SCDB merge;
- CSI cases without a corresponding SCDB case;
- the number of cases meeting each CSI threshold;
- the number of equal divisions;
- the number of cases with calculable raw and margin-normalized phi;
- the size of the principal contested sample; and
- the number and proportion of strict appointing-party splits.

Because no single CSI threshold is privileged, threshold counts are calculated for every value contained in `csi_cutoffs`.

Created objects

- **`unmatched_csi_cases`** — CSI cases for which no matching SCDB case-level record was found.
- **`phi_summary`** — descriptive statistics for raw and margin-normalized phi in the principal contested sample.
- **`threshold_diagnostics`** — case counts at each CSI threshold in both the full merged universe and principal contested sample.
- **`merge_diagnostics`** — compact summary of key merge and coding diagnostics.

```
# --- QA: merge coverage and outcome distribution -----  
  
# -----  
# CSI cases unmatched to SCDB  
# -----  
  
unmatched_csi_cases <- SCOTUS_CaseData %>%  
  dplyr::filter(is.na(term_scdb)) %>%  
  dplyr::select(caseId, term_csi, csi, csi_source)
```

```

# -----
# Phi summary in the principal contested sample
# -----

phi_summary <- SCOTUS_Contested %>% dplyr::summarise(

  n = dplyr::n(),

  mean_phi = mean(phi_abs, na.rm = TRUE),

  sd_phi = stats::sd(phi_abs, na.rm = TRUE),

  median_phi = stats::median(phi_abs, na.rm = TRUE),

  min_phi = min(phi_abs, na.rm = TRUE),

  max_phi = max(phi_abs, na.rm = TRUE),

  mean_phi_normalized = mean(phi_normalized, na.rm = TRUE),

  median_phi_normalized = stats::median(phi_normalized, na.rm = TRUE),

  strict_n = sum(strict_party_split, na.rm = TRUE),

  strict_rate = mean(strict_party_split, na.rm = TRUE))

# -----
# CSI threshold diagnostics
# -----

threshold_diagnostics <- purrr::map_dfr(
  csi_cutoffs, function(k) {

    tibble::tibble(
      CSI_cutoff = k,

      Full_universe_n = sum(SCOTUS_CaseData$csi >= k, na.rm = TRUE),

      Contested_n = sum(SCOTUS_Contested$csi >= k, na.rm = TRUE),

      Contested_share = mean(SCOTUS_Contested$csi >= k, na.rm = TRUE))

  })

```

```

# -----
# General merge diagnostics
# -----

merge_diagnostics <- tibble::tibble(

  Diagnostic = c(
    paste0(
      "CSI cases, OT", analysis_start_term, "-OT", analysis_end_term),
      "CSI cases unmatched to SCDB", "Equal-division cases",
      "Cases with calculable |phi|", "Cases with calculable normalized |phi|",
      "Main contested cases (2+ minority votes)",
      "Strict appointing-party splits in main sample"),

  N = c(
    nrow(SCOTUS_CaseData),

    nrow(unmatched_csi_cases),

    sum(SCOTUS_CaseData$equal_division, na.rm = TRUE),

    sum(!is.na(SCOTUS_CaseData$phi_abs)),

    sum(!is.na(SCOTUS_CaseData$phi_normalized)),

    nrow(SCOTUS_Contested),

    sum(SCOTUS_Contested$strict_party_split, na.rm = TRUE)))

# -----
# Display diagnostics
# -----

knitr::kable(
  merge_diagnostics, caption = "Merge and coding diagnostics",
  col.names = c("Diagnostic", "N"), align = c("l", "r"), booktabs = TRUE
)

```

Table 1: Merge and coding diagnostics

Diagnostic	N
CSI cases, OT1953-OT2024	7604
CSI cases unmatched to SCDB	4
Equal-division cases	69
Cases with calculable $ \phi $	4608
Cases with calculable normalized $ \phi $	4608
Main contested cases (2+ minority votes)	3753
Strict appointing-party splits in main sample	125

```
knitr::kable(
  threshold_diagnostics, caption = "CSI threshold diagnostics",
  col.names = c(
    "CSI cutoff", "Cases in merged universe", "Cases in contested sample",
    "Share of contested sample"), digits = 3, booktabs = TRUE)
```

Table 2: CSI threshold diagnostics

CSI cutoff	Cases in merged universe	Cases in contested sample	Share of contested sample
3	3458	2006	0.535
4	2449	1478	0.394
5	1497	967	0.258
6	950	625	0.167
7	505	345	0.092

```
print(phi_summary)
```

```
# A tibble: 1 x 10
      n mean_phi sd_phi median_phi min_phi max_phi mean_phi_normalized
<int>   <dbl> <dbl>     <dbl>   <dbl>   <dbl>         <dbl>
1  3753   0.357  0.234     0.333     0     1           0.477
# i 3 more variables: median_phi_normalized <dbl>, strict_n <int>,
#   strict_rate <dbl>
```

```
# -----
# Print unmatched cases when present
# -----
```

```

if (nrow(unmatched_csi_cases) > 0) {

  cat("\nCSI cases unmatched to SCDB:\n")

  print(unmatched_csi_cases)
}

```

CSI cases unmatched to SCDB:

A tibble: 4 x 4

	caseId	term_csi	csi	csi_source
	<chr>	<int>	<dbl>	<chr>
1	1953-031	1953	2	Collins & Cooper 1953-2014
2	1954-097	1954	2	Collins & Cooper 1953-2014
3	1999-006	1999	0	Collins & Cooper 1953-2014
4	2014-020	2014	2	Collins & Cooper 1953-2014

Descriptive Analysis

Descriptive Analysis Setup

Purpose

The following chunk defines helper functions and the centered rolling-window terms used throughout the descriptive analysis.

The principal rolling analyses use the five-Supreme-Court-term window defined in the Setup section. With a five-term centered window, the value associated with term `tt` pools cases from:

$$t - 2, t - 1, t, t + 1, t + 2$$

Using a centered window reduces term-to-term instability while preserving the long-run temporal pattern. Because a complete centered window cannot be calculated at the beginning or end of the analysis period, the rolling series begins at OT1955 and ends at OT2022.

Helper functions are also defined for means, medians, and proportions so that groups containing only missing observations return `NA` rather than `NaN`.

Created objects

- **safe_mean()** — calculates a mean while safely handling groups containing only missing observations.
- **safe_median()** — calculates a median while safely handling groups containing only missing observations.
- **safe_rate()** — calculates a proportion from a logical variable while safely handling missing observations.
- **rolling_half_window** — number of terms on each side of the center term.
- **window_centers** — center terms for which a complete rolling window is available.

```
# --- Descriptive-analysis helpers -----  
  
safe_mean <- function(x) {  
  x <- x[!is.na(x)]  
  
  if (length(x) == 0) {
```

```

    return(NA_real_)
  }

  mean(x)
}

safe_median <- function(x) {

  x <- x[!is.na(x)]

  if (length(x) == 0) {
    return(NA_real_)
  }

  stats::median(x)
}

safe_rate <- function(x) {

  safe_mean(as.numeric(x))
}

# -----
# Rolling-window definition
# -----

stopifnot(rolling_window_terms %% 2 == 1)

rolling_half_window <- as.integer((rolling_window_terms - 1) / 2)

window_centers <- seq.int(
  analysis_start_term + rolling_half_window,
  analysis_end_term - rolling_half_window)

message(
  "Centered rolling series: OT", min(window_centers), "-OT",
  max(window_centers), " using ", rolling_window_terms,
  " Supreme Court terms per window.")

```

Centered rolling series: OT1955-OT2022 using 5 Supreme Court terms per window.

Historical Strict Appointing-Party Splits

Purpose

The following code provides an early descriptive overview of the historical development of strict appointing-party alignment among highly salient contested Supreme Court cases.

The analysis restricts attention to cases with **CSI** ≥ 5 . From OT1955 through OT2004, cases are summarized in consecutive ten-term periods. Beginning with OT2005, the analysis switches to five-term periods in order to provide greater temporal resolution during the period in which the breakpoint analysis indicates that the relationship between salience and appointing-party alignment began to change.

For each period, the table reports the number of contested cases for which a strict appointing-party split can be classified, the number producing such a split, and the corresponding strict-split rate. These periods are descriptive summaries and do not replace Natural Courts or individual Supreme Court terms as the principal temporal units elsewhere in the analysis.

Created objects

- **strict_split_period_levels** — ordered historical periods used in the table.
- **Strict_Split_Period** — analytical summary of strict appointing-party splits among CSI ≥ 5 contested cases by historical period.
- **Strict_Split_Period_Table** — publication-facing version of the historical summary.

```
# --- Historical strict appointing-party splits -----  
  
# -----  
# Historical period structure  
#  
# Ten-term periods are used through OT2004:  
#  
#   OT1955-OT1964  
#   OT1965-OT1974  
#   OT1975-OT1984  
#   OT1985-OT1994  
#   OT1995-OT2004  
#  
# Five-term periods are used beginning with OT2005:  
#  
#   OT2005-OT2009  
#   OT2010-OT2014
```

```

# OT2015-OT2019
# OT2020-OT2024
#
# OT1953-OT1954 remain in the full analytical dataset but are intentionally
# outside this descriptive period table.
# -----

strict_split_period_levels <- c(
  "1955-1964", "1965-1974", "1975-1984", "1985-1994", "1995-2004",
  "2005-2009", "2010-2014", "2015-2019", "2020-2024")

# -----
# Historical strict-split summary
#
# CSI >= 5 is retained because this table summarizes strict appointing-party
# splitting specifically among highly salient contested cases.
# -----

Strict_Split_Period <- SCOTUS_Contested %>% dplyr::filter(
  term >= 1955, term <= 2024, csi >= 5, !is.na(strict_party_split)) %>%
  dplyr::mutate(

    Period = dplyr::case_when(

      term >= 1955 & term <= 1964 ~ "1955-1964",

      term >= 1965 & term <= 1974 ~ "1965-1974",

      term >= 1975 & term <= 1984 ~ "1975-1984",

      term >= 1985 & term <= 1994 ~ "1985-1994",

      term >= 1995 & term <= 2004 ~ "1995-2004",

      term >= 2005 & term <= 2009 ~ "2005-2009",

      term >= 2010 & term <= 2014 ~ "2010-2014",

      term >= 2015 & term <= 2019 ~ "2015-2019",

      term >= 2020 & term <= 2024 ~ "2020-2024",

```

```

    TRUE ~ NA_character_),

  Period = factor(Period, levels = strict_split_period_levels)) %>%
dplyr::filter(!is.na(Period)) %>% dplyr::group_by(Period) %>%
dplyr::summarise(

  N_Cases = dplyr::n(),

  Strict_Split_N = sum(strict_party_split, na.rm = TRUE),

  Strict_Split_Rate = safe_rate(strict_party_split),

  .groups = "drop")

# -----
# Publication-facing table
# -----

Strict_Split_Period_Table <- Strict_Split_Period %>% dplyr::transmute(

  Period = as.character(Period),

  `Contested cases` = N_Cases,

  `Strict party splits` = Strict_Split_N,

  `Strict split rate` =
    scales::percent(Strict_Split_Rate, accuracy = 0.1))

knitr::kable(
  Strict_Split_Period_Table, caption =
    "Strict appointing-party splits among CSI >= 5 contested cases",
  booktabs = TRUE, align = c("l", "r", "r", "r"))

```

Table 3: Strict appointing-party splits among CSI ≥ 5 contested cases

Period	Contested cases	Strict party splits	Strict split rate
1955-1964	128	0	0.0%
1965-1974	177	0	0.0%
1975-1984	196	1	0.5%
1985-1994	196	0	0.0%

Period	Contested cases	Strict party splits	Strict split rate
1995-2004	123	0	0.0%
2005-2009	42	2	4.8%
2010-2014	28	10	35.7%
2015-2019	27	6	22.2%
2020-2024	32	20	62.5%

Natural Court Composition and Historical Periods

Purpose

Describe the historical sequence of Supreme Court compositions over OT1953–OT2024.

Rather than divide the Court’s history into arbitrary calendar decades, the principal descriptive analysis uses the SCDB **naturalCourt** identifier. A Natural Court represents a period during which the membership of the Supreme Court remains unchanged.

For each Natural Court, the analysis records:

- the first and last Supreme Court terms represented;
- the first and last decision dates observed in the CSI–SCDB data;
- the Chief Justice;
- the number of Republican-appointed and Democratic-appointed justices;
- the number of CSI cases;
- the number and proportion of contested cases;
- the number and proportion of closely divided cases.

A separate transition object compares the membership of successive Natural Courts so that later analysis can identify which justices entered or departed when a new Court composition began.

Created objects

- **NaturalCourt_Periods** — term and decision-date boundaries for each Natural Court.
- **NaturalCourt_Members** — one row per justice $\times$ Natural Court.
- **NaturalCourt_Composition** — appointing-party composition of each Natural Court.
- **NaturalCourt_MemberSets** — list of justice membership sets for each Natural Court.

- **NaturalCourt_Transitions** — justices entering and departing between successive Natural Courts.
- **NaturalCourt_CaseSummary** — CSI case counts and vote-division proportions within each Natural Court.

```
# --- Natural Court composition and historical periods -----
# -----
# Parse SCDB decision dates
# -----

parse_scdb_date <- function(x) {

  if (inherits(x, "Date")) {
    return(x)
  }

  as.Date(x, format = "%m/%d/%Y")
}

# -----
# Natural Court period boundaries
# -----

NaturalCourt_Periods <- SCDB_JusticeVotes %>%
  dplyr::filter(!is.na(naturalCourt)) %>%
  dplyr::mutate(decision_date_parsed = parse_scdb_date(dateDecision)) %>%
  dplyr::group_by(naturalCourt) %>% dplyr::summarise(

    First_Term = min(term, na.rm = TRUE),

    Last_Term = max(term, na.rm = TRUE),

    First_Decision = if (
      all(is.na(decision_date_parsed))) {
        as.Date(NA)
      } else {
        min(decision_date_parsed, na.rm = TRUE)
      },

    Last_Decision = if (
      all(is.na(decision_date_parsed))) {
```

```

      as.Date(NA)
    } else {
      max(decision_date_parsed, na.rm = TRUE)
    },

    Chief =
      paste(sort(unique(stats::na.omit(chief))), collapse = " / "),

    .groups = "drop")

# -----
# Justice membership within each Natural Court
# -----

NaturalCourt_Members <- SCDB_JusticeVotes %>% dplyr::filter(
  !is.na(naturalCourt), !is.na(justiceName), !is.na(appointing_party)
) %>% dplyr::distinct(naturalCourt, justiceName, appointing_party)

NaturalCourt_Composition <- NaturalCourt_Members %>%
  dplyr::group_by(naturalCourt) %>% dplyr::summarise(

    N_Justices = dplyr::n_distinct(justiceName),

    R_Appointed =
      dplyr::n_distinct(justiceName[appointing_party == "R"]),

    D_Appointed =
      dplyr::n_distinct(justiceName[appointing_party == "D"]),

    .groups = "drop")

# -----
# Membership transitions between successive Natural Courts
# -----

NaturalCourt_MemberSets <- NaturalCourt_Members %>%
  dplyr::group_by(naturalCourt) %>% dplyr::summarise(
    Members = list(sort(unique(justiceName))), .groups = "drop") %>%
  dplyr::left_join(
    NaturalCourt_Periods %>%
      dplyr::select(naturalCourt, First_Term, First_Decision),
    by = "naturalCourt") %>%

```

```

dplyr::arrange(First_Term, First_Decision, naturalCourt)

NaturalCourt_Transitions <- purrr::map_dfr(
  seq_len(nrow(NaturalCourt_MemberSets)), function(i) {

    current_members <- NaturalCourt_MemberSets$Members[[i]]

    previous_members <- if (i == 1) {
      character(0)
    } else {
      NaturalCourt_MemberSets$Members[[i - 1]]
    }

    entered <- setdiff(current_members, previous_members)

    departed <- setdiff(previous_members, current_members)

    tibble::tibble(
      naturalCourt = NaturalCourt_MemberSets$naturalCourt[i],

      Entered = if (i == 1) {
        "Initial composition"
      } else if (length(entered) == 0) {
        ""
      } else {
        paste(entered, collapse = ", ")
      },

      Departed = if (i == 1) {
        ""
      } else if (length(departed) == 0) {
        ""
      } else {
        paste(departed, collapse = ", ")
      })
  })

# -----
# CSI case and vote-division counts within Natural Courts
# -----

NaturalCourt_CaseSummary <- SCOTUS_CaseData %>%

```

```
dplyr::filter(!is.na(naturalCourt)) %>%
dplyr::group_by(naturalCourt) %>% dplyr::summarise(

  N_CSI_Cases = dplyr::n(),

  N_Vote_Classifiable = sum(!is.na(minVotes)),

  Unanimous_Rate = safe_rate(minVotes == 0),

  One_Minority_Rate = safe_rate(minVotes == 1),

  Contested_2plus_Rate = safe_rate(minVotes >= 2),

  Closely_Divided_3plus_Rate = safe_rate(minVotes >= 3),

  .groups = "drop")
```

Appointing-Party Alignment by Natural Court

Purpose

Summarize appointing-party alignment within each Supreme Court composition.

For every Natural Court, the table reports the principal raw absolute phi coefficient and the margin-normalized phi coefficient side by side. Reporting both measures shows whether the historical pattern is sensitive to Court composition or vote margins.

The table additionally reports the appointing-party composition of the Court, the number of CSI cases, the number of contested cases included in the alignment analysis, and the proportion of contested cases producing a strict appointing-party split.

Internal identifiers and variables needed only for subsequent analysis are retained in the underlying data objects but omitted from the displayed table.

Created objects

- **NaturalCourt_Alignment** — appointing-party alignment statistics among contested cases by Natural Court.
- **NaturalCourt_Descriptives** — combined Natural Court composition, vote-division, and alignment data used for the table and subsequent analyses.

```

# --- Appointing-party alignment by Natural Court -----

NaturalCourt_Alignment <- SCOTUS_Contested %>%
  dplyr::filter(!is.na(naturalCourt)) %>%
  dplyr::group_by(naturalCourt) %>% dplyr::summarise(

    N_Contested_Analytic = dplyr::n(),

    Mean_Phi = safe_mean(phi_abs),

    Mean_Phi_Normalized = safe_mean(phi_normalized),

    Median_Phi = safe_median(phi_abs),

    Strict_Split_N = sum(strict_party_split, na.rm = TRUE),

    Strict_Split_Rate = safe_rate(strict_party_split),

    .groups = "drop")

NaturalCourt_Descriptives <- NaturalCourt_Periods %>%
  dplyr::left_join(NaturalCourt_Composition, by = "naturalCourt") %>%
  dplyr::left_join(NaturalCourt_Transitions, by = "naturalCourt") %>%
  dplyr::left_join(NaturalCourt_CaseSummary, by = "naturalCourt") %>%
  dplyr::left_join(NaturalCourt_Alignment, by = "naturalCourt") %>%
  dplyr::arrange(First_Term, First_Decision, naturalCourt) %>%
  dplyr::mutate(

    Court_Index = dplyr::row_number(),

    Court_Period = dplyr::if_else(
      First_Term == Last_Term, paste0("OT", First_Term),
      paste0("OT", First_Term, "-OT", Last_Term)))

knitr::kable(
  NaturalCourt_Descriptives %>% dplyr::transmute(

    Period = Court_Period,

    Chief = Chief,

    `Appointment composition` =

```

```

paste0(R_Appointed, "R-", D_Appointed, "D"),

`CSI cases` = N_CSI_Cases,

Contested = N_Contested_Analytic,

`Mean |phi|` = round(Mean_Phi, 3),

`Norm. |phi|` = round(Mean_Phi_Normalized, 3),

`Strict split` = scales::percent(Strict_Split_Rate, accuracy = 0.1)
), caption =
paste0(
  "Supreme Court composition and appointing-party alignment ",
  "by Natural Court"
),
booktabs = TRUE, align = c("l", "l", "c", "r", "r", "r", "r", "r"))

```

Table 4: Supreme Court composition and appointing-party alignment by Natural Court

Period	Chief	Appointment composition	CSI cases	Contested	Mean phi	Norm. phi	Strict split
OT1953	Warren	1R-8D	84	43	0.281	0.564	0.0%
OT1954	Warren	1R-7D	50	18	0.257	0.453	0.0%
OT1954- OT1956	Warren	2R-7D	145	76	0.254	0.378	0.0%
OT1956	Warren	3R-6D	48	26	0.236	0.269	0.0%
OT1956- OT1958	Warren	4R-5D	204	113	0.217	0.287	0.0%
OT1958- OT1961	Warren	5R-4D	408	223	0.244	0.318	0.0%
OT1961	Warren	4R-5D	57	25	0.312	0.417	0.0%
OT1962- OT1964	Warren	4R-5D	365	149	0.297	0.423	0.0%
OT1965- OT1966	Warren	4R-5D	214	99	0.317	0.437	0.0%
OT1967- OT1968	Warren	4R-5D	198	81	0.284	0.405	0.0%
OT1968	Warren	4R-4D	40	15	0.327	0.464	0.0%
OT1969	Burger	4R-4D	89	47	0.311	0.451	0.0%

Period	Chief	Appointment composition	CSI cases	Contested	Mean $ \phi $	Norm. $ \phi $	Strict split
OT1969- OT1970	Burger	5R-4D	150	72	0.371	0.452	0.0%
OT1971	Burger	4R-3D	21	9	0.212	0.245	0.0%
OT1971- OT1975	Burger	6R-3D	576	294	0.395	0.457	0.0%
OT1975	Burger	6R-2D	5	1	0.333	0.333	0.0%
OT1975- OT1980	Burger	7R-2D	815	445	0.323	0.427	0.2%
OT1981- OT1985	Burger	7R-2D	747	371	0.279	0.381	0.5%
OT1986	Rehnquist	7R-2D	161	104	0.204	0.267	0.0%
OT1987	Rehnquist	6R-2D	26	7	0.202	0.238	0.0%
OT1987- OT1989	Rehnquist	7R-2D	418	221	0.230	0.301	0.0%
OT1990	Rehnquist	7R-2D	129	66	0.320	0.426	0.0%
OT1991- OT1992	Rehnquist	8R-1D	239	114	0.305	0.623	0.0%
OT1993	Rehnquist	8R-1D	96	47	0.285	0.571	0.0%
OT1994- OT2004	Rehnquist	7R-2D	970	440	0.409	0.582	1.1%
OT2005	Roberts	7R-2D	20	7	0.483	0.604	14.3%
OT2005- OT2008	Roberts	7R-2D	271	149	0.411	0.592	2.7%
OT2009	Roberts	6R-3D	76	36	0.466	0.576	2.8%
OT2009- OT2015	Roberts	5R-4D	371	168	0.643	0.743	20.2%
OT2015- OT2016	Roberts	4R-4D	77	29	0.507	0.753	3.4%
OT2016- OT2018	Roberts	6R-4D	106	50	0.722	0.873	34.0%
OT2018- OT2019	Roberts	5R-4D	123	65	0.662	0.792	23.1%
OT2020- OT2021	Roberts	6R-3D	120	58	0.736	0.812	37.9%
OT2022- OT2024	Roberts	6R-3D	181	85	0.633	0.708	25.9%

Vote-Division Proportions by Case Salience

Purpose

The following code describes how frequently cases produce substantial disagreement before conditioning the principal analysis on contested cases.

This distinguishes two questions:

1. Does case salience relate to whether the Court divides at all?
2. Conditional on substantial disagreement, does the division align with appointing-president party?

For each CSI cutoff, cases at or above the cutoff are compared with cases below the cutoff. The table reports:

- unanimous decisions;
- decisions with exactly one minority vote;
- contested decisions with at least two minority votes;
- closely divided decisions with at least three minority votes.

The latter two categories are intentionally overlapping: every case with at least three minority votes is also a contested case.

Created objects

- **Vote_Division_Thresholds** — vote-division proportions for high- and lower-salience cases at every CSI cutoff.

```
# --- Vote-division proportions across CSI thresholds -----  
  
Vote_Division_Thresholds <- purrr::map_dfr(  
  csi_cutoffs, function(k) {  
  
    SCOTUS_CaseData %>% dplyr::filter(!is.na(minVotes)) %>% dplyr::mutate(  
  
      Salience_Group =  
        dplyr::if_else(csi >= k, "At or above cutoff", "Below cutoff")  
    ) %>% dplyr::group_by(Salience_Group) %>% dplyr::summarise(  
  
      N_Cases = dplyr::n(),
```

```

    Unanimous_Rate = safe_rate(minVotes == 0),

    One_Minority_Rate = safe_rate(minVotes == 1),

    Contested_2plus_Rate = safe_rate(minVotes >= 2),

    Closely_Divided_3plus_Rate = safe_rate(minVotes >= 3),

    .groups = "drop") %>% dplyr::mutate(CSI_Cutoff = k)
}) %>% dplyr::mutate(
  Salience_Group = factor(
    Salience_Group, levels = c("Below cutoff", "At or above cutoff"))
) %>% dplyr::arrange(CSI_Cutoff, Salience_Group) %>%
  dplyr::select(CSI_Cutoff, Salience_Group, dplyr::everything())

knitr::kable(
  Vote_Division_Thresholds %>% dplyr::transmute(

    `CSI cutoff` = CSI_Cutoff,

    Group = dplyr::if_else(
      Salience_Group == "At or above cutoff",
      paste0("CSI >= ", CSI_Cutoff), paste0("CSI < ", CSI_Cutoff)),

    Cases = N_Cases,

    Unanimous = scales::percent(Unanimous_Rate, accuracy = 0.1),

    `One minority` = scales::percent(One_Minority_Rate, accuracy = 0.1),

    `Contested (2+)` =
      scales::percent(Contested_2plus_Rate, accuracy = 0.1),

    `Closely divided (3+)` =
      scales::percent(Closely_Divided_3plus_Rate, accuracy = 0.1)),
  caption = "Vote-division proportions by CSI threshold", booktabs = TRUE,
  align = c("r", "l", "r", "r", "r", "r", "r", "r"))

```

Table 5: Vote-division proportions by CSI threshold

CSI cutoff	Group	Cases	Unanimous	One minority	Contested (2+)	Closely divided (3+)
3	CSI < 3	4142	45.1%	11.5%	43.4%	29.1%
3	CSI >= 3	3458	30.4%	10.9%	58.7%	43.3%
4	CSI < 4	5151	43.3%	11.4%	45.3%	31.0%
4	CSI >= 4	2449	28.0%	11.0%	61.0%	45.3%
5	CSI < 5	6103	41.8%	11.5%	46.7%	32.2%
5	CSI >= 5	1497	24.4%	10.4%	65.2%	49.3%
6	CSI < 6	6650	40.6%	11.3%	48.1%	33.3%
6	CSI >= 6	950	22.9%	10.6%	66.4%	51.4%
7	CSI < 7	7095	39.5%	11.4%	49.0%	34.3%
7	CSI >= 7	505	22.2%	8.9%	68.9%	54.3%

Five-Term Rolling Appointing-Party Alignment Across CSI Thresholds

Purpose

The following code calculates trailing rolling appointing-party alignment for an exact **CSI = 0** reference group and for nested CSI thresholds from **CSI >= 0 through CSI >= 7**.

CSI = 0 isolates contested cases receiving a total CSI score of exactly zero. **CSI >= 0** represents the complete contested CSI case universe, while successively higher thresholds show how the historical pattern changes as increasingly low-salience cases are excluded.

For presentation, rolling estimates are displayed only when at least five contested cases contribute to the relevant series and window. This restriction is applied **only to the figure**: all calculated estimates remain unchanged in the underlying analytical objects. The same minimum-sample rule is applied uniformly to every displayed series.

All nine trajectories use identical line widths and are ordered in the legend from **CSI = 0** through **CSI >= 7**. The colors progress from cooler tones at the low-salience end toward warmer tones and dark red at the highest threshold. A second version of the figure adds the existing Natural Court composition shading.

Created objects

- **csi_plot_cutoffs** — nested plotting thresholds from 0 through 7.
- **Rolling_Phi_Thresholds** — rolling estimates for $\text{CSI} \geq 0$ through $\text{CSI} \geq 7$.
- **Rolling_Phi_ExactZero** — rolling estimates restricted to $\text{CSI} = 0$.
- **min_cases_for_plot** — minimum number of contested cases required to display a rolling estimate.
- **Rolling_Phi_Thresholds_Long** — long-format nested-threshold estimates.
- **Rolling_Phi_ExactZero_Long** — long-format exact-zero estimates.
- **Rolling_Phi_Plot_Long** — combined plotting data with the minimum-sample display rule applied.
- **csi_plot_colors** — ordered color palette for all nine series.
- **NaturalCourt_Shading** — Natural Court background-band data.
- **p_phi_thresholds** — rolling alignment figure without Court-composition shading.
- **p_phi_thresholds_shaded** — corresponding figure with Natural Court composition shading.

```
# --- Five-term rolling alignment across CSI thresholds -----  
  
# -----  
# Plotting thresholds  
# -----  
  
csi_plot_cutoffs <- 0:7  
  
# -----  
# Rolling years  
#  
# 1953 = 1953  
# 1954 = 1953-1954  
# 1955 = 1953-1955  
# ...  
# 1957 = 1953-1957  
# 1958 = 1954-1958  
#  
# This allows the series to span the complete 1953-2024 analysis period.  
# -----
```

```

rolling_years <- seq.int(analysis_start_term, analysis_end_term)

rolling_lookback <- rolling_window_terms - 1L

# -----
# Rolling estimates across nested CSI thresholds 0-7
# -----

Rolling_Phi_Thresholds <- purrr::map_dfr(
  csi_plot_cutoffs, function(k) {

    purrr::map_dfr(
      rolling_years, function(t) {

        window_start <- max(analysis_start_term, t - rolling_lookback)

        x <- SCOTUS_Contested %>%
          dplyr::filter(term >= window_start, term <= t, csi >= k)

        tibble::tibble(

          Year = t,

          Window_Start = window_start,

          Window_End = t,

          N_Terms = t - window_start + 1L,

          CSI_Cutoff = k,

          N_Cases = nrow(x),

          Mean_Phi = safe_mean(x$phi_abs),

          Mean_Phi_Normalized = safe_mean(x$phi_normalized),

          Strict_Split_N = sum(x$strict_party_split, na.rm = TRUE),

          Strict_Split_Rate = safe_rate(x$strict_party_split))
      })
  })

```

```

# -----
# Rolling estimates for cases with CSI exactly equal to zero
# -----

Rolling_Phi_ExactZero <- purrr::map_dfr(
  rolling_years, function(t) {

    window_start <- max(analysis_start_term, t - rolling_lookback)

    x <- SCOTUS_Contested %>%
      dplyr::filter(term >= window_start, term <= t, csi == 0)

    tibble::tibble(

      Year = t,

      Window_Start = window_start,

      Window_End = t,

      N_Terms = t - window_start + 1L,

      N_Cases = nrow(x),

      Mean_Phi = safe_mean(x$phi_abs),

      Mean_Phi_Normalized = safe_mean(x$phi_normalized),

      Strict_Split_N = sum(x$strict_party_split, na.rm = TRUE),

      Strict_Split_Rate = safe_rate(x$strict_party_split))
  })

# -----
# Minimum sample size for displayed rolling estimates
#
# For plotting only, a rolling estimate is displayed when at least five
# contested cases contribute to the relevant series and window.
#
# The same rule is applied uniformly to all nine plotted series.
# -----

```

```

min_cases_for_plot <- 5L

# -----
# Reshape nested threshold estimates for plotting
# -----

Rolling_Phi_Thresholds_Long <- Rolling_Phi_Thresholds %>% dplyr::select(
  Year, CSI_Cutoff, N_Cases, Mean_Phi, Mean_Phi_Normalized) %>%
tidyr::pivot_longer(
  cols = c(Mean_Phi, Mean_Phi_Normalized), names_to =
    "Alignment_Measure", values_to = "Mean_Alignment") %>% dplyr::mutate(

  Alignment_Measure = dplyr::recode(
    Alignment_Measure, Mean_Phi = "Mean absolute phi",
    Mean_Phi_Normalized = "Mean margin-normalized phi"),

  Series = paste0("CSI >= ", CSI_Cutoff))

# -----
# Reshape exact-zero reference series for plotting
# -----

Rolling_Phi_ExactZero_Long <- Rolling_Phi_ExactZero %>%
dplyr::select(Year, N_Cases, Mean_Phi, Mean_Phi_Normalized) %>%
tidyr::pivot_longer(
  cols = c(Mean_Phi, Mean_Phi_Normalized), names_to =
    "Alignment_Measure", values_to = "Mean_Alignment") %>% dplyr::mutate(

  Alignment_Measure = dplyr::recode(
    Alignment_Measure, Mean_Phi = "Mean absolute phi",
    Mean_Phi_Normalized = "Mean margin-normalized phi"),

  Series = "CSI = 0")

# -----
# Combine all nine plotting series
# -----

csi_series_levels <- c("CSI = 0", paste0("CSI >= ", 0:7))

Rolling_Phi_Plot_Long <- dplyr::bind_rows(
  Rolling_Phi_ExactZero_Long, Rolling_Phi_Thresholds_Long) %>%

```

```

dplyr::mutate(

  Alignment_Measure = factor(
    Alignment_Measure,
    levels = c("Mean absolute phi", "Mean margin-normalized phi")),

  Series = factor(Series, levels = csi_series_levels),

  Eligible_For_Plot = N_Cases >= min_cases_for_plot &
    !is.na(Mean_Alignment),

  Mean_Alignment_Plot =
    dplyr::if_else(Eligible_For_Plot, Mean_Alignment, NA_real_) %>%
dplyr::group_by(Alignment_Measure, Series) %>%
dplyr::arrange(Year, .by_group = TRUE) %>% dplyr::mutate(

  Plot_Run = cumsum(
    Eligible_For_Plot != dplyr::lag(
      Eligible_For_Plot, default = dplyr::first(Eligible_For_Plot))
)) %>% dplyr::ungroup()

# -----
# Series colours
# -----

csi_plot_colors <- c(
  "CSI = 0" = "#5E4FA2", "CSI >= 0" = "#3288BD", "CSI >= 1" = "#66C2A5",
  "CSI >= 2" = "#8FCB82", "CSI >= 3" = "#B8C95A", "CSI >= 4" = "#D9B43B",
  "CSI >= 5" = "#E9962F", "CSI >= 6" = "#E85B3A", "CSI >= 7" = "#9E0142")

# -----
# Natural Court shading
#
# Background bands are clipped to exactly the same horizontal period as
# the plotted series: 1953-2024.
#
# Red = more Republican-appointed justices
# Blue = more Democratic-appointed justices
#
# Larger appointing-party margins receive slightly stronger shading.
# -----

```

```

NaturalCourt_Shading <- NaturalCourt_Descriptives %>%
  dplyr::arrange(First_Term, First_Decision, naturalCourt) %>%
  dplyr::transmute(

    naturalCourt,

    xmin = pmax(First_Term - 0.5, analysis_start_term),

    xmax = pmin>Last_Term + 0.5, analysis_end_term),

    Balance = R_Appointed - D_Appointed,

    Shade_Colour = dplyr::case_when(
      Balance > 0 ~ "#D95F5F", Balance < 0 ~ "#5B8FD1",
      TRUE      ~ "#FFFFFF"),

    Shade_Intensity = dplyr::case_when(
      Balance == 0 ~ 0, TRUE ~
        pmin(0.12, 0.035 + 0.015 * abs(Balance)))) %>%
  dplyr::filter(xmax > xmin)

# -----
# Shared plot scales and theme
# -----

phi_threshold_scales <- list(

  ggplot2::facet_wrap(~ Alignment_Measure, ncol = 1),

  ggplot2::scale_color_manual(
    values = csi_plot_colors, breaks = csi_series_levels, labels =
      csi_series_levels, name = NULL, drop = FALSE),

  ggplot2::guides(
    color = ggplot2::guide_legend(
      nrow = 2, byrow = TRUE,
      override.aes = list(linewidth = 0.62, alpha = 1))),

  ggplot2::scale_x_continuous(
    limits = c(analysis_start_term, analysis_end_term),
    breaks = seq(1960, 2020, by = 10),
    expand = ggplot2::expansion(mult = c(0, 0))),

```

```

ggplot2::scale_y_continuous(
  limits = c(0, 1), breaks = seq(0, 1, by = 0.2),
  expand = ggplot2::expansion(mult = c(0, 0.02))),

ggplot2::labs(
  title = "Voting patterns by appointing party in Supreme Court cases",
  x = "Year", y = "Phi coefficient"),

ggplot2::theme_minimal(),

ggplot2::theme(

  legend.position = "bottom",

  legend.direction = "horizontal",

  legend.box = "horizontal",

  panel.grid.minor = ggplot2::element_blank(),

  panel.grid.major.x = ggplot2::element_blank(),

  panel.spacing = grid::unit(0.8, "lines"),

  strip.text = ggplot2::element_text(face = "bold", hjust = 0.5),

  plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),

  axis.title.x =
    ggplot2::element_text(margin = ggplot2::margin(t = 10)),

  axis.title.y =
    ggplot2::element_text(margin = ggplot2::margin(r = 10)),

  legend.margin = ggplot2::margin(t = 5)))

# -----
# A. Figure without Natural Court shading
# -----

p_phi_thresholds <- ggplot2::ggplot() +

```

```

ggplot2::geom_line(
  data = Rolling_Phi_Plot_Long, ggplot2::aes(
    x = Year, y = Mean_Alignment_Plot, group =
      interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

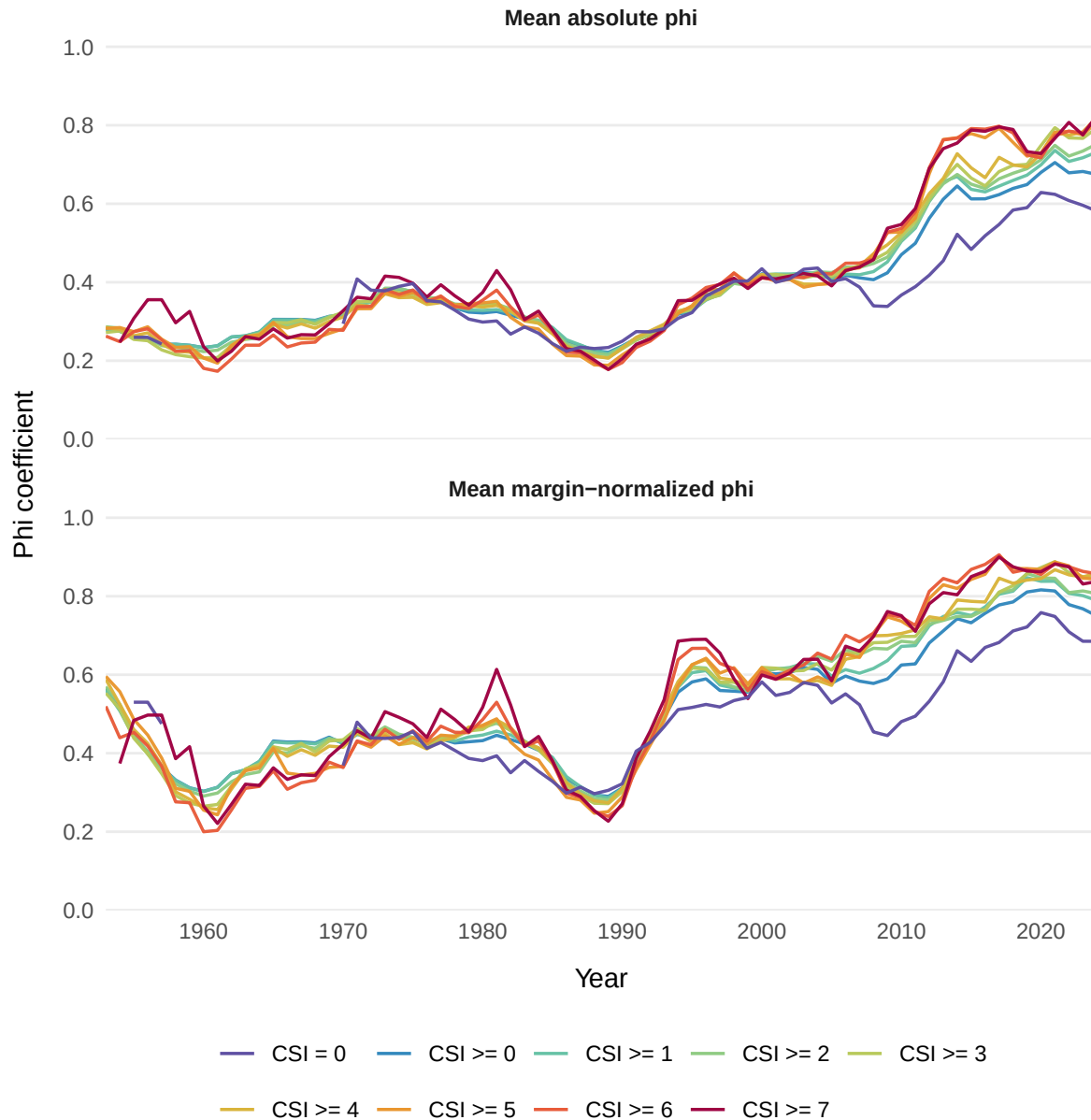
ggplot2::geom_line(
  data = Rolling_Phi_Plot_Long, ggplot2::aes(
    x = Year, y = Mean_Alignment_Plot, group =
      interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

phi_threshold_scales

print(p_phi_thresholds)

```

Voting patterns by appointing party in Supreme Court cases



```
ggplot2::ggsave(
  filename = file.path(fig_dir, "01_phi_rolling_csi_thresholds.emf"),
  plot = p_phi_thresholds, device = devEMF::emf, width = 6.27, height = 6.8,
  units = "in")
```

```

# B. Figure with Natural Court composition shading
# -----

p_phi_thresholds_shaded <- ggplot2::ggplot() +

  ggplot2::geom_rect(
    data = NaturalCourt_Shading, ggplot2::aes(
      xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf,
      fill = Shade_Colour, alpha = Shade_Intensity), inherit.aes = FALSE
  ) +

  ggplot2::scale_fill_identity(guide = "none") +

  ggplot2::scale_alpha_identity(guide = "none") +

  ggplot2::geom_line(
    data = Rolling_Phi_Plot_Long, ggplot2::aes(
      x = Year, y = Mean_Alignment_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

  ggplot2::geom_line(
    data = Rolling_Phi_Plot_Long, ggplot2::aes(
      x = Year, y = Mean_Alignment_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

  phi_threshold_scales

print(p_phi_thresholds_shaded)

```

Voting patterns by appointing party in Supreme Court cases



```
ggplot2::ggsave(
  filename = file.path(
    fig_dir, "01_phi_rolling_csi_thresholds_shaded.emf"), plot =
    p_phi_thresholds_shaded, device = devEMF::emf, width = 6.27, height =
    6.8, units = "in")
```

High- Versus Lower-Salience Cases Across CSI Thresholds

Purpose

The following code calculates five-term rolling appointing-party alignment separately for cases above and below each CSI threshold.

Earlier versions of the analysis plotted the high- and low-salience series directly, but that figure has been superseded by the subsequent **salience-gap analysis**, which presents the same comparison more directly as the difference between the two groups. These estimates are nonetheless retained because they are used to calculate the subsequent salience gaps.

Created objects

- **Rolling_Phi_HighLow** — five-term rolling estimates of raw absolute phi, margin-normalized phi, and strict appointing-party split rates separately for cases above and below each CSI cutoff.

```
# --- Five-term rolling high-versus-low salience comparison -----

Rolling_Phi_HighLow <- purrr::map_dfr(
  csi_cutoffs, function(k) {

    purrr::map_dfr(
      window_centers, function(t) {

        SCOTUS_Contested %>% dplyr::filter(
          term >= t - rolling_half_window, term <= t + rolling_half_window
        ) %>% dplyr::mutate(

          Salience_Group = dplyr::if_else(csi >= k, "High", "Low")) %>%
          dplyr::group_by(Salience_Group) %>% dplyr::summarise(

            N_Cases = dplyr::n(),

            Mean_Phi = safe_mean(phi_abs),

            Mean_Phi_Normalized = safe_mean(phi_normalized),

            Strict_Split_Rate = safe_rate(strict_party_split),

            .groups = "drop") %>%
```

```

    dplyr::mutate(Center_Term = t, CSI_Cutoff = k)
  })
})

```

Five-Term Rolling Saliency-Band Gaps

Purpose

The following code calculates trailing rolling saliency gaps using **four mutually exclusive CSI bands** rather than the earlier series of nested cutoffs.

Cases with **CSI < 2** form the common low-saliency reference group. The remaining cases are grouped into **CSI 2–4**, **CSI 5–6**, and **CSI ≥ 7**. For each rolling window, the figure reports the difference in appointing-party alignment between each higher-saliency band and the CSI < 2 reference group.

The horizontal zero line therefore represents the alignment level of the CSI < 2 reference category. Positive values indicate greater appointing-party alignment than among low-saliency cases, while negative values indicate less alignment.

CSI < 2 is preferred to CSI < 1 because the latter would consist only of exact-zero CSI cases, which can be sparse and produce unstable rolling estimates. As elsewhere, estimates are displayed only when both the reference and comparison groups contain at least five contested cases.

Raw absolute phi is shown in the upper panel and margin-normalized phi in the lower panel. Both unshaded and Natural Court-shaded versions are produced.

Created objects

- **saliency_band_levels** — four mutually exclusive CSI saliency bands.
- **Rolling_Phi_Bands** — trailing rolling alignment estimates for each band.
- **Rolling_Phi_BandGaps** — rolling high-saliency-minus-reference gaps.
- **Rolling_Phi_BandGaps_Long** — long-format plotting data.
- **band_gap_series_levels** — ordered plotting labels for the three non-reference bands.
- **band_gap_colors** — colors for the three substantive trajectories.
- **p_phi_band_gap** — unshaded saliency-gap figure.
- **p_phi_band_gap_shaded** — corresponding Natural Court-shaded figure.

```

# --- Five-term rolling salience-band gaps -----

rolling_years <- seq.int(analysis_start_term, analysis_end_term)

rolling_lookback <- rolling_window_terms - 1L

# -----
# Minimum sample size for displayed rolling estimates
# -----

min_cases_for_plot <- 5L

# -----
# Mutually exclusive salience bands
#
# CSI < 2 is the common low-salience reference group.
# -----

salience_band_levels <- c("CSI < 2", "CSI 2-4", "CSI 5-6", "CSI >= 7")

# -----
# Rolling appointing-party alignment by salience band
# -----

Rolling_Phi_Bands <- purrr::map_dfr(
  rolling_years, function(t) {

    window_start <- max(analysis_start_term, t - rolling_lookback)

    x <- SCOTUS_Contested %>%
      dplyr::filter(term >= window_start, term <= t) %>% dplyr::mutate(

        Salience_Band = dplyr::case_when(
          csi < 2 ~ "CSI < 2",
          csi >= 2 & csi < 5 ~ "CSI 2-4",
          csi >= 5 & csi < 7 ~ "CSI 5-6",
          csi >= 7 ~ "CSI >= 7",
          TRUE ~ NA_character_)

    band_summary <- x %>% dplyr::filter(!is.na(Salience_Band)) %>%
      dplyr::group_by(Salience_Band) %>% dplyr::summarise(

```

```

    N_Cases = dplyr::n(),

    Mean_Phi = safe_mean(phi_abs),

    Mean_Phi_Normalized = safe_mean(phi_normalized),

    Strict_Split_Rate = safe_rate(strict_party_split),

    .groups = "drop")

tibble::tibble(Salience_Band = salience_band_levels) %>%
  dplyr::left_join(band_summary, by = "Salience_Band") %>%
  dplyr::mutate(

    N_Cases = tidyr::replace_na(N_Cases, 0L),

    Year = t,

    Window_Start = window_start,

    Window_End = t,

    N_Terms = t - window_start + 1L)
})

# -----
# Convert band estimates to wide form
# -----

Rolling_Phi_BandGaps <- Rolling_Phi_Bands %>% dplyr::mutate(

  Band_ID = dplyr::case_when(
    Salience_Band == "CSI < 2" ~ "lt2",
    Salience_Band == "CSI 2-4" ~ "b24",
    Salience_Band == "CSI 5-6" ~ "b56",
    Salience_Band == "CSI >= 7" ~ "ge7",
    TRUE ~ NA_character_) %>% dplyr::select(
    Year, Window_Start, Window_End, N_Terms, Band_ID, N_Cases, Mean_Phi,
    Mean_Phi_Normalized, Strict_Split_Rate) %>% tidyr::pivot_wider(
    names_from = Band_ID, values_from = c(
      N_Cases, Mean_Phi, Mean_Phi_Normalized, Strict_Split_Rate))

```

```

Rolling_Phi_BandGaps_Long <- dplyr::bind_rows(

# CSI 2-4 relative to CSI < 2
Rolling_Phi_BandGaps %>% dplyr::transmute(

  Year, Window_Start, Window_End, N_Terms,

  Series = "CSI 2-4",

  N_Cases_Reference = N_Cases_lt2,

  N_Cases_Comparison = N_Cases_b24,

  Eligible_For_Plot = N_Cases_lt2 >= min_cases_for_plot &
    N_Cases_b24 >= min_cases_for_plot,

  Phi_Gap = Mean_Phi_b24 - Mean_Phi_lt2,

  Phi_Normalized_Gap = Mean_Phi_Normalized_b24 -
    Mean_Phi_Normalized_lt2,

  Strict_Split_Gap = Strict_Split_Rate_b24 - Strict_Split_Rate_lt2),

# CSI 5-6 relative to CSI < 2
Rolling_Phi_BandGaps %>% dplyr::transmute(

  Year, Window_Start, Window_End, N_Terms,

  Series = "CSI 5-6",

  N_Cases_Reference = N_Cases_lt2,

  N_Cases_Comparison = N_Cases_b56,

  Eligible_For_Plot = N_Cases_lt2 >= min_cases_for_plot &
    N_Cases_b56 >= min_cases_for_plot,

  Phi_Gap = Mean_Phi_b56 - Mean_Phi_lt2,

  Phi_Normalized_Gap = Mean_Phi_Normalized_b56 -
    Mean_Phi_Normalized_lt2,

```

```

    Strict_Split_Gap = Strict_Split_Rate_b56 - Strict_Split_Rate_lt2),

Rolling_Phi_BandGaps %>% dplyr::transmute(

    Year, Window_Start, Window_End, N_Terms,

    Series = "CSI >= 7",

    N_Cases_Reference = N_Cases_lt2,

    N_Cases_Comparison = N_Cases_ge7,

    Eligible_For_Plot = N_Cases_lt2 >= min_cases_for_plot &
      N_Cases_ge7 >= min_cases_for_plot,

    Phi_Gap = Mean_Phi_ge7 - Mean_Phi_lt2,

    Phi_Normalized_Gap = Mean_Phi_Normalized_ge7 -
      Mean_Phi_Normalized_lt2,

    Strict_Split_Gap = Strict_Split_Rate_ge7 - Strict_Split_Rate_lt2)
) %>% tidyr::pivot_longer(
  cols = c(Phi_Gap, Phi_Normalized_Gap), names_to = "Gap_Measure",
  values_to = "Gap_Value") %>% dplyr::mutate(

  Gap_Measure = dplyr::recode(
    Gap_Measure, Phi_Gap = "Mean absolute phi gap", Phi_Normalized_Gap =
      "Mean margin-normalized phi gap"),

  Gap_Measure = factor(
    Gap_Measure, levels = c(
      "Mean absolute phi gap", "Mean margin-normalized phi gap")),

  Gap_Value_Plot = dplyr::if_else(
    Eligible_For_Plot & !is.na(Gap_Value), Gap_Value, NA_real_)

# -----
# Series ordering
# -----

band_gap_series_levels <- c("CSI 2-4", "CSI 5-6", "CSI >= 7")

```

```

Rolling_Phi_BandGaps_Long <- Rolling_Phi_BandGaps_Long %>% dplyr::mutate(

  Series = factor(Series, levels = band_gap_series_levels)) %>%
dplyr::group_by(Gap_Measure, Series) %>%
dplyr::arrange(Year, .by_group = TRUE) %>% dplyr::mutate(

  Gap_Available = !is.na(Gap_Value_Plot),

  Plot_Run = cumsum(
    Gap_Available != dplyr::lag(
      Gap_Available, default = dplyr::first(Gap_Available)))) %>%
dplyr::ungroup()

# -----
# Salience-band colors
# -----

band_gap_colors <- c(
  "CSI 2-4" = "#B8C95A", "CSI 5-6" = "#E9962F", "CSI >= 7" = "#9E0142")

# -----
# Shared salience-gap scales and theme
# -----

phi_band_gap_scales <- list(

  ggplot2::facet_wrap(~ Gap_Measure, ncol = 1),

  ggplot2::scale_color_manual(
    values = band_gap_colors, breaks = band_gap_series_levels, labels =
      band_gap_series_levels, name = NULL, drop = FALSE),

  ggplot2::guides(
    color = ggplot2::guide_legend(
      nrow = 1, byrow = TRUE, override.aes =
        list(linewidth = 0.62, alpha = 1))),

  ggplot2::scale_x_continuous(
    limits = c(analysis_start_term, analysis_end_term),
    breaks = seq(1960, 2020, by = 10),
    expand = ggplot2::expansion(mult = c(0, 0))),

```

```

ggplot2::scale_y_continuous(
  breaks = c(-0.3, -0.15, 0, 0.15, 0.3),
  expand = ggplot2::expansion(mult = c(0.02, 0.02))),

ggplot2::coord_cartesian(ylim = c(-0.3, 0.3)),

ggplot2::labs(
  title = "Salience gaps in appointing-party alignment", x = "Year", y =
    "Difference from CSI < 2 in phi coefficient"),

ggplot2::theme_minimal(),

ggplot2::theme(

  legend.position = "bottom",

  legend.direction = "horizontal",

  legend.box = "horizontal",

  panel.grid.minor = ggplot2::element_blank(),

  panel.grid.major.x = ggplot2::element_blank(),

  panel.spacing = grid::unit(0.8, "lines"),

  strip.text = ggplot2::element_text(face = "bold", hjust = 0.5),

  plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),

  axis.title.x =
    ggplot2::element_text(margin = ggplot2::margin(t = 10)),

  axis.title.y =
    ggplot2::element_text(margin = ggplot2::margin(r = 10)),

  legend.margin = ggplot2::margin(t = 5)))

# -----
# A. Salience-band gap figure without Natural Court shading
# -----

```

```

p_phi_band_gap <- ggplot2::ggplot() +

  ggplot2::geom_hline(
    yintercept = 0, color = "grey55", linewidth = 0.30) +

  ggplot2::geom_line(
    data = Rolling_Phi_BandGaps_Long, ggplot2::aes(
      x = Year, y = Gap_Value_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

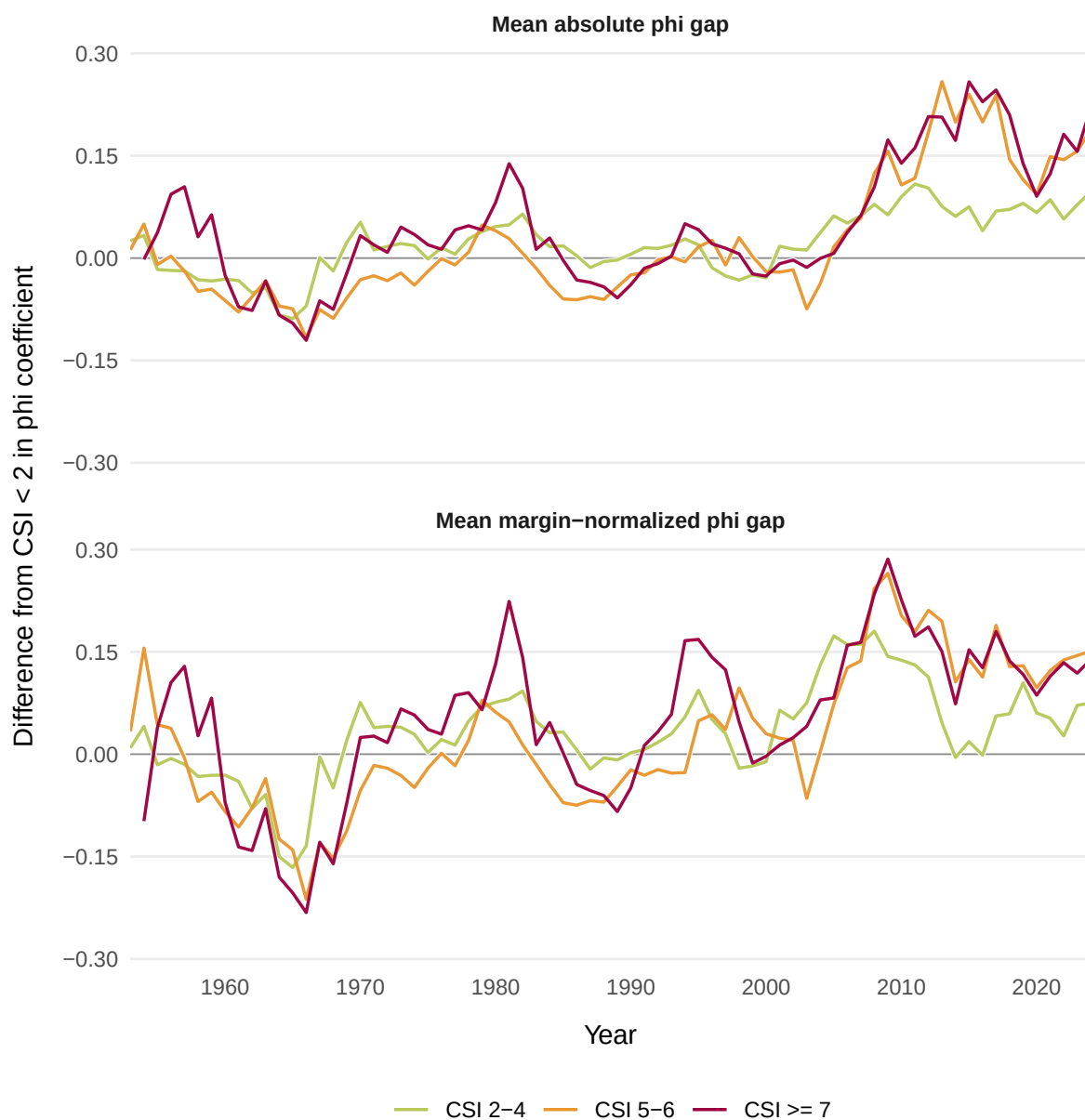
  ggplot2::geom_line(
    data = Rolling_Phi_BandGaps_Long, ggplot2::aes(
      x = Year, y = Gap_Value_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

  phi_band_gap_scales

print(p_phi_band_gap)

```

Saliency gaps in appointing-party alignment



```
ggplot2::ggsave(
  filename = file.path(fig_dir, "02_phi_rolling_saliency_band_gap.emf"),
  plot = p_phi_band_gap, device = devEMF::emf, width = 6.27, height = 6.8,
  units = "in")
```

```

# B. Salience-band gap figure with Natural Court composition shading
# -----

p_phi_band_gap_shaded <- ggplot2::ggplot() +

  ggplot2::geom_rect(
    data = NaturalCourt_Shading, ggplot2::aes(
      xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf,
      fill = Shade_Colour, alpha = Shade_Intensity), inherit.aes = FALSE
  ) +

  ggplot2::scale_fill_identity(guide = "none") +

  ggplot2::scale_alpha_identity(guide = "none") +

  ggplot2::geom_hline(
    yintercept = 0, color = "grey55", linewidth = 0.30) +

  ggplot2::geom_line(
    data = Rolling_Phi_BandGaps_Long, ggplot2::aes(
      x = Year, y = Gap_Value_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

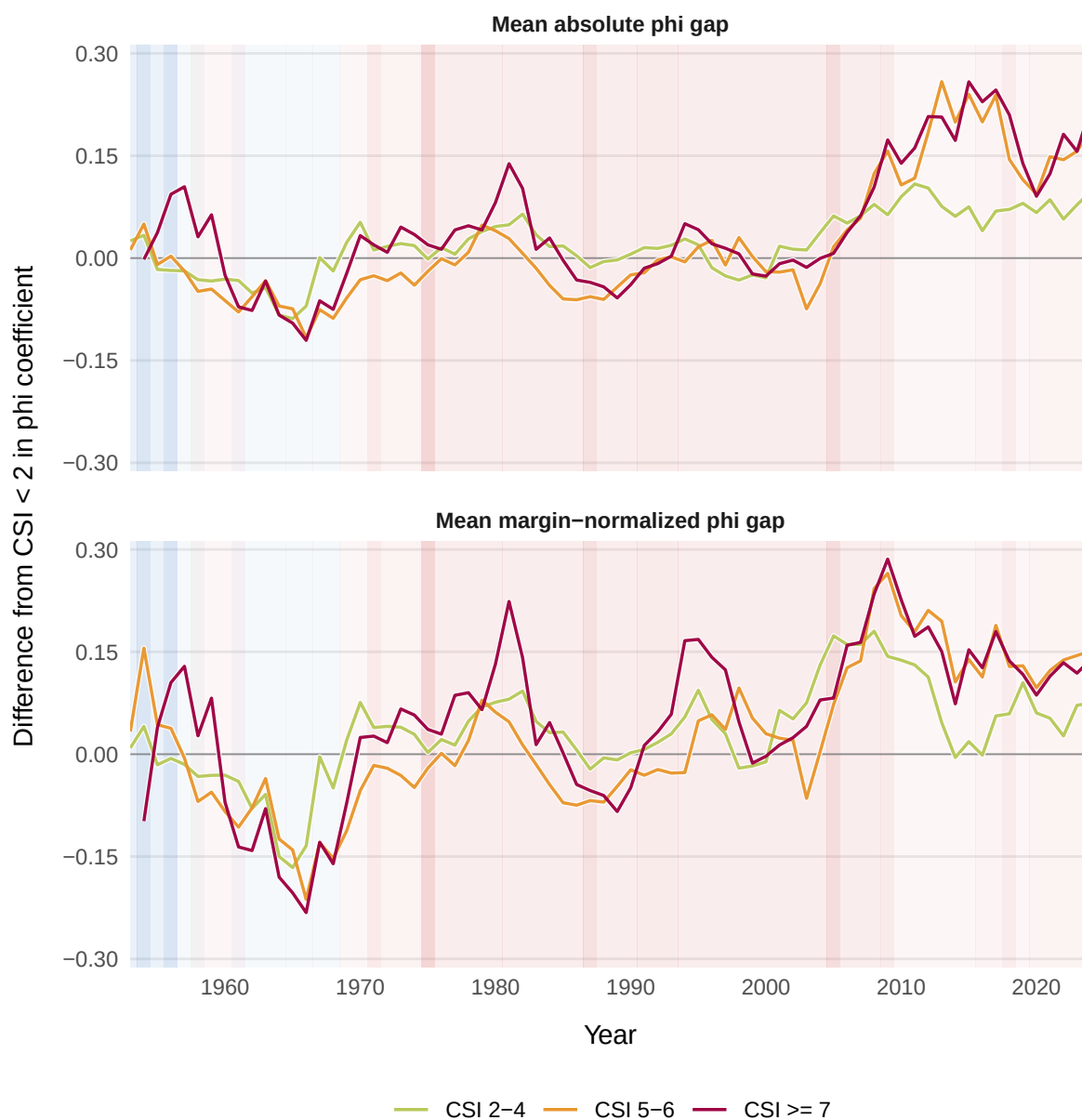
  ggplot2::geom_line(
    data = Rolling_Phi_BandGaps_Long, ggplot2::aes(
      x = Year, y = Gap_Value_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

  phi_band_gap_scales

print(p_phi_band_gap_shaded)

```

Saliency gaps in appointing-party alignment



```
ggplot2::ggsave(
  filename = file.path(
    fig_dir, "02_phi_rolling_saliency_band_gap_shaded.emf"), plot =
    p_phi_band_gap_shaded, device = devEMF::emf, width = 6.27, height = 6.8,
    units = "in")
```

Pre-/Post-2010 Descriptive Comparison

Purpose

The following code presents the descriptive comparison of appointing-party alignment before and after OT2010 in publication-ready form.

The first table reports the underlying alignment levels for high- and lower-salience contested cases at each CSI cutoff. Raw absolute phi and margin-normalized phi are presented side by side, together with the number of cases and the strict appointing-party split rate.

The second table reports the high-minus-low salience gap. To keep the table narrow and readable, the three outcome measures are placed in a long format, with pre-2010, post-2010, and change columns rather than a very wide set of machine-style variable names.

Created objects

- **PrePost_Phi_Table** — reader-facing table of high- and lower-salience alignment levels before and after 2010.
- **PrePost_Gaps_Table** — reader-facing table of pre-2010, post-2010, and change-in-gap estimates for raw phi, normalized phi, and strict appointing-party splits.

```
# --- Pre-/post-2010 descriptive comparison -----  
  
PrePost_Phi <- purrr::map_dfr(  
  csi_cutoffs, function(k) {  
  
    SCOTUS_Contested %>% dplyr::mutate(  
  
      Era = dplyr::if_else(term <= 2009, "Pre2010", "Post2010"),  
  
      Salience_Group = dplyr::if_else(csi >= k, "High", "Low")) %>%  
    dplyr::group_by(Era, Salience_Group) %>% dplyr::summarise(  
  
      N_Cases = dplyr::n(),  
  
      Mean_Phi = safe_mean(phi_abs),  
  
      Mean_Phi_Normalized = safe_mean(phi_normalized),  
  
      Strict_Split_Rate = safe_rate(strict_party_split),
```

```

    .groups = "drop") %>% dplyr::mutate(CSI_Cutoff = k)
  })

PrePost_Gaps <- PrePost_Phi %>% dplyr::select(
  CSI_Cutoff, Era, Salience_Group, N_Cases, Mean_Phi, Mean_Phi_Normalized,
  Strict_Split_Rate) %>% tidyr::pivot_wider(
  names_from = Salience_Group, values_from = c(
    N_Cases, Mean_Phi, Mean_Phi_Normalized, Strict_Split_Rate)) %>%
dplyr::mutate(

  Phi_Gap = Mean_Phi_High - Mean_Phi_Low,

  Phi_Normalized_Gap = Mean_Phi_Normalized_High - Mean_Phi_Normalized_Low,

  Strict_Split_Gap = Strict_Split_Rate_High - Strict_Split_Rate_Low) %>%
dplyr::select(
  CSI_Cutoff, Era, Phi_Gap, Phi_Normalized_Gap, Strict_Split_Gap) %>%
tidyr::pivot_wider(
  names_from = Era,
  values_from = c(Phi_Gap, Phi_Normalized_Gap, Strict_Split_Gap)) %>%
dplyr::mutate(

  Change_Phi_Gap = Phi_Gap_Post2010 - Phi_Gap_Pre2010,

  Change_Phi_Normalized_Gap = Phi_Normalized_Gap_Post2010 -
    Phi_Normalized_Gap_Pre2010,

  Change_Strict_Split_Gap = Strict_Split_Gap_Post2010 -
    Strict_Split_Gap_Pre2010)

# -----
# Publication table: underlying high- and lower-salience estimates
# -----

PrePost_Phi_Table <- PrePost_Phi %>% dplyr::mutate(

  Cutoff = paste0("CSI >= ", CSI_Cutoff),

  Period =
    dplyr::recode(Era, Pre2010 = "Pre-2010", Post2010 = "Post-2010"),

  Group = dplyr::if_else(

```

```

    Saliency_Group == "High", paste0("CSI >= ", CSI_Cutoff),
    paste0("CSI < ", CSI_Cutoff))) %>% dplyr::transmute(

    Cutoff,

    Period,

    Group,

    Cases = N_Cases,

    `Mean |phi|` = round(Mean_Phi, 3),

    `Normalized |phi|` = round(Mean_Phi_Normalized, 3),

    `Strict split` = scales::percent(Strict_Split_Rate, accuracy = 0.1))

knitr::kable(
  PrePost_Phi_Table, caption =
    "Appointing-party alignment before and after OT2010", booktabs = TRUE,
  align = c("l", "l", "l", "r", "r", "r", "r"))

```

Table 6: Appointing-party alignment before and after OT2010

Cutoff	Period	Group	Cases	Mean phi	Normalized phi	Strict split
CSI >= 3	Post- 2010	CSI >= 3	195	0.726	0.818	35.4%
CSI >= 3	Post- 2010	CSI < 3	260	0.603	0.729	16.2%
CSI >= 3	Pre- 2010	CSI >= 3	1811	0.312	0.436	0.3%
CSI >= 3	Pre- 2010	CSI < 3	1487	0.321	0.438	0.5%
CSI >= 4	Post- 2010	CSI >= 4	136	0.745	0.827	39.0%
CSI >= 4	Post- 2010	CSI < 4	319	0.618	0.741	18.2%
CSI >= 4	Pre- 2010	CSI >= 4	1342	0.312	0.439	0.4%
CSI >= 4	Pre- 2010	CSI < 4	1956	0.319	0.435	0.4%

Cutoff	Period	Group	Cases	Mean phi	Normalized phi	Strict split
CSI ≥ 5	Post- 2010	CSI ≥ 5	87	0.769	0.844	41.4%
CSI ≥ 5	Post- 2010	CSI < 5	368	0.629	0.749	20.4%
CSI ≥ 5	Pre- 2010	CSI ≥ 5	880	0.310	0.433	0.3%
CSI ≥ 5	Pre- 2010	CSI < 5	2418	0.319	0.438	0.5%
CSI ≥ 6	Post- 2010	CSI ≥ 6	66	0.778	0.854	45.5%
CSI ≥ 6	Post- 2010	CSI < 6	389	0.635	0.752	20.8%
CSI ≥ 6	Pre- 2010	CSI ≥ 6	559	0.320	0.450	0.4%
CSI ≥ 6	Pre- 2010	CSI < 6	2739	0.316	0.434	0.4%
CSI ≥ 7	Post- 2010	CSI ≥ 7	44	0.773	0.835	43.2%
CSI ≥ 7	Post- 2010	CSI < 7	411	0.643	0.760	22.4%
CSI ≥ 7	Pre- 2010	CSI ≥ 7	301	0.335	0.472	0.3%
CSI ≥ 7	Pre- 2010	CSI < 7	2997	0.314	0.433	0.4%

```

PrePost_Gaps_Table <- PrePost_Gaps %>% dplyr::select(
  CSI_Cutoff, Phi_Gap_Pre2010, Phi_Gap_Post2010, Change_Phi_Gap,
  Phi_Normalized_Gap_Pre2010, Phi_Normalized_Gap_Post2010,
  Change_Phi_Normalized_Gap, Strict_Split_Gap_Pre2010,
  Strict_Split_Gap_Post2010, Change_Strict_Split_Gap) %>%
tidyr::pivot_longer(
  cols = -CSI_Cutoff, names_to = "Statistic", values_to = "Value") %>%
dplyr::mutate(

  Measure = dplyr::case_when(
    stringr::str_detect(
      Statistic, "Phi_Normalized") ~ "Normalized |phi| gap",

    stringr::str_detect(
      Statistic, "Strict_Split") ~ "Strict-split gap",

```

```

    TRUE ~ "Mean |phi| gap"),

  Period = dplyr::case_when(
    stringr::str_detect(Statistic, "Pre2010") ~ "Pre-2010",

    stringr::str_detect(Statistic, "Post2010") ~ "Post-2010",

    stringr::str_detect(Statistic, "^Change") ~ "Change")) %>%
dplyr::select(CSI_Cutoff, Measure, Period, Value) %>%
tidyr::pivot_wider(names_from = Period, values_from = Value) %>%
dplyr::mutate(

  Cutoff = paste0("CSI >= ", CSI_Cutoff),

  Measure = factor(
    Measure, levels = c(
      "Mean |phi| gap", "Normalized |phi| gap", "Strict-split gap"))
) %>% dplyr::arrange(CSI_Cutoff, Measure) %>% dplyr::transmute(

  Cutoff,

  Measure = as.character(Measure),

  `Pre-2010` = round(`Pre-2010`, 3),

  `Post-2010` = round(`Post-2010`, 3),

  Change = round(Change, 3))

knitr::kable(
  PrePost_Gaps_Table, caption =
    "Change in the high-minus-low salience gap after OT2010", booktabs =
    TRUE, align = c("l", "l", "r", "r", "r"))

```

Table 7: Change in the high-minus-low salience gap after OT2010

Cutoff	Measure	Pre-2010	Post-2010	Change
CSI >= 3	Mean phi gap	-0.009	0.122	0.131
CSI >= 3	Normalized phi gap	-0.001	0.089	0.091
CSI >= 3	Strict-split gap	-0.002	0.192	0.194
CSI >= 4	Mean phi gap	-0.007	0.128	0.135

Cutoff	Measure	Pre-2010	Post-2010	Change
CSI ≥ 4	Normalized $ \phi $ gap	0.004	0.086	0.082
CSI ≥ 4	Strict-split gap	0.000	0.208	0.208
CSI ≥ 5	Mean $ \phi $ gap	-0.009	0.140	0.149
CSI ≥ 5	Normalized $ \phi $ gap	-0.005	0.095	0.100
CSI ≥ 5	Strict-split gap	-0.001	0.210	0.211
CSI ≥ 6	Mean $ \phi $ gap	0.004	0.143	0.139
CSI ≥ 6	Normalized $ \phi $ gap	0.015	0.101	0.086
CSI ≥ 6	Strict-split gap	-0.001	0.246	0.247
CSI ≥ 7	Mean $ \phi $ gap	0.021	0.130	0.109
CSI ≥ 7	Normalized $ \phi $ gap	0.039	0.075	0.037
CSI ≥ 7	Strict-split gap	-0.001	0.208	0.209

Estimated Breakpoint in the Saliency–Alignment Relationship

Purpose

The following code estimates the Supreme Court term at which the relationship between case saliency and appointing-party alignment changes most sharply.

The breakpoint search is conducted on the underlying contested-case data rather than on the five-term rolling series. This avoids using overlapping rolling windows to determine the timing of the change.

For every candidate breakpoint from OT1990 through OT2018, the analysis estimates the relationship between continuous CSI and appointing-party alignment before and after the candidate term while absorbing term-specific differences in the overall level of alignment. The selected breakpoint is the term producing the lowest residual sum of squares.

The principal search uses raw absolute ϕ . The procedure is repeated independently using margin-normalized ϕ as a robustness check.

Uncertainty in the location of the breakpoint is estimated using a **term-cluster bootstrap**. Supreme Court terms rather than individual cases are resampled, preserving the within-term dependence structure of the data. The resulting bootstrap distribution provides a 95% interval for the estimated breakpoint.

This data-driven breakpoint is distinct from the theoretically motivated OT2010 division used in the preceding descriptive table. The data-driven estimate can then be compared with the pre-specified OT2010 division.

Created objects

- **break_candidates** — candidate breakpoint terms searched by the procedure.
- **Breakpoint_Data_Raw** — within-term transformed case-level data for raw absolute phi.
- **Breakpoint_Data_Normalized** — corresponding data for margin-normalized phi.
- **Breakpoint_Search_Raw** — raw-phi breakpoint search results for every candidate term.
- **Breakpoint_Search_Normalized** — normalized-phi breakpoint search results.
- **Breakpoint_Bootstrap** — term-cluster bootstrap distribution of estimated breakpoint terms.
- **Breakpoint_Summary** — point estimates, bootstrap intervals, and pre-/post-break CSI slopes.
- **Breakpoint_Table** — publication-facing summary table with reader-friendly column names.

```
# --- Estimated breakpoint in the salience-alignment relationship -----  
  
# -----  
# Breakpoint-search parameters  
#  
# The candidate range is intentionally broader than OT2010 so that the  
# data-driven search is not mechanically constrained to reproduce the  
# theoretically motivated breakpoint.  
# -----  
  
break_search_start <- 1990L  
  
break_search_end <- 2018L  
  
break_candidates <- seq.int(break_search_start, break_search_end)  
  
# -----  
# Number of term-cluster bootstrap replications  
# -----  
  
break_boot_reps <- 1000L  
  
# -----
```

```

# Prepare data for breakpoint estimation
#
# The transformation below removes the mean of both CSI and the outcome
# separately within each Supreme Court term.
#
# This is algebraically equivalent to including a fixed effect for every
# Supreme Court term in the breakpoint regression.
#
# The resulting breakpoint therefore concerns a change in the relationship
# between salience and appointing-party alignment rather than merely a change
# in the overall level of Court polarization over time.
# -----

prepare_breakpoint_data <- function(
  data, outcome) {

  data %>%
    dplyr::filter(!is.na(term), !is.na(csi), !is.na(.data[[outcome]])) %>%
    dplyr::group_by(term) %>% dplyr::mutate(

      CSI_Within = csi - mean(csi, na.rm = TRUE),

      Outcome_Within = .data[[outcome]] -
        mean(.data[[outcome]], na.rm = TRUE)) %>% dplyr::ungroup() %>%
    dplyr::filter(is.finite(CSI_Within), is.finite(Outcome_Within))
}

Breakpoint_Data_Raw <- prepare_breakpoint_data(
  SCOTUS_Contested, "phi_abs")

Breakpoint_Data_Normalized <- prepare_breakpoint_data(
  SCOTUS_Contested, "phi_normalized")

# -----
# Restrict raw and normalized analyses to the same set of Supreme Court terms
#
# This keeps the timing comparison between the two outcome measures directly
# comparable.
# -----

common_break_terms <- intersect(
  unique(Breakpoint_Data_Raw$term),

```

```

unique(Breakpoint_Data_Normalized$term)) %>% sort()

Breakpoint_Data_Raw <- Breakpoint_Data_Raw %>%
  dplyr::filter(term %in% common_break_terms)

Breakpoint_Data_Normalized <- Breakpoint_Data_Normalized %>%
  dplyr::filter(term %in% common_break_terms)

# -----
# Breakpoint-search function
#
# For every candidate term b:
#
#   Pre-break CSI slope
#     = relationship between CSI and alignment before b
#
#   Slope change
#     = change in the CSI relationship beginning in b
#
#   Post-break CSI slope
#     = pre-break slope + slope change
#
# The best breakpoint is the candidate producing the smallest residual
# sum of squares.
# -----

search_breakpoint <- function(
  data, candidates, weights = NULL) {

  if (
    is.null(weights)) {

    weights <- rep(1, nrow(data))
  }

  search_results <- purrr::map_dfr(
    candidates, function(b) {

      d <- data %>% dplyr::mutate(

        Post_Break = as.integer(term >= b),

```

```

    CSI_Post = CSI_Within * Post_Break)

fit <- stats::lm(
  Outcome_Within ~ 0 + CSI_Within + CSI_Post, data = d, weights =
    weights)

coefficients <- stats::coef(fit)

pre_slope <- unname(coefficients["CSI_Within"])

slope_change <- unname(coefficients["CSI_Post"])

post_slope <- pre_slope + slope_change

rss <- sum(weights * stats::residuals(fit)^2, na.rm = TRUE)

tibble::tibble(

  Break_Term = b,

  RSS = rss,

  Pre_CSI_Slope = pre_slope,

  Slope_Change = slope_change,

  Post_CSI_Slope = post_slope)
})

search_results %>%
  dplyr::mutate(Delta_RSS = RSS - min(RSS, na.rm = TRUE)) %>%
  dplyr::arrange(Break_Term)
}

# -----
# Primary breakpoint searches
# -----

Breakpoint_Search_Raw <- search_breakpoint(
  Breakpoint_Data_Raw, break_candidates)

Breakpoint_Search_Normalized <- search_breakpoint(

```

```

Breakpoint_Data_Normalized, break_candidates)

# -----
# Identify the best-fitting breakpoint
#
# If two candidates were ever exactly tied, the earlier term would be
# selected. Exact ties are unlikely in practice.
# -----

Breakpoint_Best_Raw <- Breakpoint_Search_Raw %>%
  dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1)

Breakpoint_Best_Normalized <- Breakpoint_Search_Normalized %>%
  dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1)

# -----
# Term-cluster bootstrap
# -----

set.seed(20260908)

Breakpoint_Bootstrap <- purrr::map_dfr(
  seq_len(break_boot_reps), function(r) {

    sampled_terms <- sample(
      common_break_terms, size = length(common_break_terms), replace =
        TRUE)

    term_frequency <- table(
      factor(sampled_terms, levels = common_break_terms))

    term_weights <- stats::setNames(
      as.numeric(term_frequency), as.character(common_break_terms))

    raw_weights <- unname(
      term_weights[
        as.character(Breakpoint_Data_Raw$term)])

    normalized_weights <- unname(
      term_weights[
        as.character(Breakpoint_Data_Normalized$term)])
  })

```

```

raw_search <- search_breakpoint(
  Breakpoint_Data_Raw, break_candidates, weights = raw_weights)

normalized_search <- search_breakpoint(
  Breakpoint_Data_Normalized, break_candidates, weights =
    normalized_weights)

raw_break <- raw_search %>% dplyr::arrange(RSS, Break_Term) %>%
  dplyr::slice(1) %>% dplyr::pull(Break_Term)

normalized_break <- normalized_search %>%
  dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1) %>%
  dplyr::pull(Break_Term)

tibble::tibble(

  Replicate = r,

  Raw_Break = raw_break,

  Normalized_Break = normalized_break)
})

# -----
# Helper for discrete bootstrap confidence intervals
# -----

breakpoint_interval <- function(
  x) {

  c(
    Lower = as.integer(
      stats::quantile(x, probs = 0.025, type = 1, na.rm = TRUE)),

    Median = as.integer(stats::median(x, na.rm = TRUE)),

    Upper = as.integer(
      stats::quantile(x, probs = 0.975, type = 1, na.rm = TRUE)))
  }

raw_break_interval <- breakpoint_interval(Breakpoint_Bootstrap$Raw_Break)

```

```

normalized_break_interval <- breakpoint_interval(
  Breakpoint_Bootstrap$Normalized_Break)

# -----
# Summarize breakpoint estimates
# -----

Breakpoint_Summary <- dplyr::bind_rows(

  tibble::tibble(

    Measure = "Mean absolute phi",

    Estimated_Break = Breakpoint_Best_Raw$Break_Term,

    Bootstrap_Median = raw_break_interval[
      "Median"],

    CI_Lower = raw_break_interval[
      "Lower"],

    CI_Upper = raw_break_interval[
      "Upper"],

    Pre_CSI_Slope = Breakpoint_Best_Raw$Pre_CSI_Slope,

    Post_CSI_Slope = Breakpoint_Best_Raw$Post_CSI_Slope,

    Slope_Change = Breakpoint_Best_Raw$Slope_Change),

  tibble::tibble(

    Measure = "Margin-normalized phi",

    Estimated_Break = Breakpoint_Best_Normalized$Break_Term,

    Bootstrap_Median = normalized_break_interval[
      "Median"],

    CI_Lower = normalized_break_interval[
      "Lower"],

```

```

CI_Upper = normalized_break_interval[
  "Upper"],

Pre_CSI_Slope = Breakpoint_Best_Normalized$Pre_CSI_Slope,

Post_CSI_Slope = Breakpoint_Best_Normalized$Post_CSI_Slope,

Slope_Change = Breakpoint_Best_Normalized$Slope_Change))

# -----
# Publication-facing breakpoint table
# -----

Breakpoint_Table <- Breakpoint_Summary %>% dplyr::transmute(

  Measure,

  `Estimated break` = paste0("OT", Estimated_Break),

  `Bootstrap median` = paste0("OT", Bootstrap_Median),

  `95% bootstrap interval` = paste0("OT", CI_Lower, "-OT", CI_Upper),

  `Pre-break CSI slope` = round(Pre_CSI_Slope, 3),

  `Post-break CSI slope` = round(Post_CSI_Slope, 3),

  `Slope change` = round(Slope_Change, 3))

knitr::kable(
  Breakpoint_Table, caption =
    paste0(
      "Estimated breakpoint in the relationship between case salience ",
      "and appointing-party alignment"
    ),
  booktabs = TRUE, align = c("l", "l", "l", "l", "r", "r", "r"))

```

Table 8: Estimated breakpoint in the relationship between case salience and appointing-party alignment

Measure	Estimated break	Bootstrap median	95% bootstrap interval	Pre-break CSI slope	Post-break CSI slope	Slope change
Mean absolute phi	OT2005	OT2005	OT2002-OT2007	-0.002	0.029	0.031
Margin-normalized phi	OT2004	OT2004	OT1990-OT2004	-0.001	0.029	0.030

```
# -----
# Five best-fitting candidate terms
# -----

Breakpoint_TopCandidates <- dplyr::bind_rows(

  Breakpoint_Search_Raw %>% dplyr::arrange(RSS) %>%
    dplyr::slice_head(n = 5) %>%
    dplyr::mutate(Measure = "Mean absolute phi"),

  Breakpoint_Search_Normalized %>% dplyr::arrange(RSS) %>%
    dplyr::slice_head(n = 5) %>%
    dplyr::mutate(Measure = "Margin-normalized phi")) %>%
  dplyr::transmute(

    Measure,

    Term = paste0("OT", Break_Term),

    `Difference from minimum RSS` = round(Delta_RSS, 4))

knitr::kable(
  Breakpoint_TopCandidates, caption =
    "Best-fitting candidate breakpoint terms", booktabs = TRUE)
```

Table 9: Best-fitting candidate breakpoint terms

Measure	Term	Difference from minimum RSS
Mean absolute phi	OT2005	0.0000
Mean absolute phi	OT2004	0.1163
Mean absolute phi	OT2007	0.2322
Mean absolute phi	OT2006	0.2325
Mean absolute phi	OT2008	0.2890
Margin-normalized phi	OT2004	0.0000
Margin-normalized phi	OT2005	0.1351
Margin-normalized phi	OT2003	0.3264
Margin-normalized phi	OT1991	0.4249
Margin-normalized phi	OT2002	0.4422

Salience Gap by Natural Court

Purpose

The following code determines whether changes in the relationship between salience and appointing-party alignment coincide with changes in Supreme Court membership.

The analysis uses the same salience-band structure as the rolling salience-gap analysis. Cases with **CSI < 2** form the common low-salience reference group, while higher-salience cases are divided into **CSI 2–4**, **CSI 5–6**, and **CSI >= 7**.

For every Natural Court, the code calculates the difference between each higher-salience band and the CSI < 2 reference group in:

- raw absolute phi;
- margin-normalized phi;
- strict appointing-party split rate.

Raw and margin-normalized phi are displayed in vertically stacked panels. Estimates are displayed only when both the reference and comparison groups contain at least five contested cases. The underlying estimates are retained regardless of sample size.

Natural Courts make it possible to compare the emerging salience gap with changes in Court membership.

Created objects

- **NaturalCourt_Salience** — appointing-party alignment statistics by Natural Court and salience band.
- **NaturalCourt_Salience_Gaps** — differences from the CSI < 2 reference group by Natural Court.
- **NaturalCourt_Salience_Gaps_Long** — long-format raw and normalized gap data used for plotting.
- **NaturalCourt_Salience_Gaps_Table** — reader-facing Natural Court gap table retained for possible appendix use but not automatically printed.
- **p_natural_court_gaps** — vertically faceted figure showing raw and normalized salience gaps across successive Natural Courts.

```

# --- Salience gap by Natural Court -----

# -----
# Salience bands
#
# CSI < 2 is the common low-salience reference.
#
#   CSI < 2
#   CSI 2-4
#   CSI 5-6
#   CSI >= 7
# -----

salience_band_levels <- c("CSI < 2", "CSI 2-4", "CSI 5-6", "CSI >= 7")

# -----
# Minimum number of cases required to display a Natural Court comparison
# -----

min_cases_natural_court <- 5L

# -----
# Alignment statistics by Natural Court and salience band
# -----

NaturalCourt_Salience <- SCOTUS_Contested %>%
  dplyr::filter(!is.na(naturalCourt)) %>% dplyr::mutate(

    Salience_Band = dplyr::case_when(
      csi < 2 ~ "CSI < 2",
      csi >= 2 & csi < 5 ~ "CSI 2-4",
      csi >= 5 & csi < 7 ~ "CSI 5-6",
      csi >= 7 ~ "CSI >= 7",
      TRUE ~ NA_character_) %>%
  dplyr::filter(!is.na(Salience_Band)) %>%
  dplyr::group_by(naturalCourt, Salience_Band) %>% dplyr::summarise(

    N_Cases = dplyr::n(),

    Mean_Phi = safe_mean(phi_abs),

    Mean_Phi_Normalized = safe_mean(phi_normalized),

```

```

Strict_Split_Rate = safe_rate(strict_party_split),

.groups = "drop") %>% tidyr::complete(
naturalCourt, Saliency_Band = saliency_band_levels, fill =
  list(N_Cases = 0L))

# -----
# Convert saliency-band estimates to wide form
# -----

NaturalCourt_Saliency_Wide <- NaturalCourt_Saliency %>% dplyr::mutate(

  Band_ID = dplyr::case_when(
    Saliency_Band == "CSI < 2" ~ "lt2",
    Saliency_Band == "CSI 2-4" ~ "b24",
    Saliency_Band == "CSI 5-6" ~ "b56",
    Saliency_Band == "CSI >= 7" ~ "ge7",
    TRUE ~ NA_character_) %>% dplyr::select(
naturalCourt, Band_ID, N_Cases, Mean_Phi, Mean_Phi_Normalized,
Strict_Split_Rate) %>% tidyr::pivot_wider(
names_from = Band_ID, values_from = c(
  N_Cases, Mean_Phi, Mean_Phi_Normalized, Strict_Split_Rate))

# -----
# Calculate differences from the CSI < 2 reference group
# -----

NaturalCourt_Saliency_Gaps <- dplyr::bind_rows(

  NaturalCourt_Saliency_Wide %>% dplyr::transmute(

    naturalCourt,

    Series = "CSI 2-4",

    N_Cases_Reference = N_Cases_lt2,

    N_Cases_Comparison = N_Cases_b24,

    Eligible_For_Plot = N_Cases_lt2 >= min_cases_natural_court &
      N_Cases_b24 >= min_cases_natural_court,

```

```

Phi_Gap = Mean_Phi_b24 - Mean_Phi_lt2,

Phi_Normalized_Gap = Mean_Phi_Normalized_b24 -
  Mean_Phi_Normalized_lt2,

Strict_Split_Gap = Strict_Split_Rate_b24 - Strict_Split_Rate_lt2),

NaturalCourt_Salience_Wide %>% dplyr::transmute(

  naturalCourt,

  Series = "CSI 5-6",

  N_Cases_Reference = N_Cases_lt2,

  N_Cases_Comparison = N_Cases_b56,

  Eligible_For_Plot = N_Cases_lt2 >= min_cases_natural_court &
    N_Cases_b56 >= min_cases_natural_court,

  Phi_Gap = Mean_Phi_b56 - Mean_Phi_lt2,

  Phi_Normalized_Gap = Mean_Phi_Normalized_b56 -
    Mean_Phi_Normalized_lt2,

  Strict_Split_Gap = Strict_Split_Rate_b56 - Strict_Split_Rate_lt2),

# CSI >= 7 relative to CSI < 2
NaturalCourt_Salience_Wide %>% dplyr::transmute(

  naturalCourt,

  Series = "CSI >= 7",

  N_Cases_Reference = N_Cases_lt2,

  N_Cases_Comparison = N_Cases_ge7,

  Eligible_For_Plot = N_Cases_lt2 >= min_cases_natural_court &
    N_Cases_ge7 >= min_cases_natural_court,

  Phi_Gap = Mean_Phi_ge7 - Mean_Phi_lt2,

```

```

    Phi_Normalized_Gap = Mean_Phi_Normalized_ge7 -
      Mean_Phi_Normalized_lt2,

    Strict_Split_Gap = Strict_Split_Rate_ge7 - Strict_Split_Rate_lt2)
) %>% dplyr::left_join(
  NaturalCourt_Descriptives %>% dplyr::select(
    naturalCourt, Court_Index, Court_Period, Chief, R_Appointed,
    D_Appointed), by = "naturalCourt") %>%
dplyr::arrange(Court_Index, Series)

# -----
# Reader-facing Natural Court table
# -----

NaturalCourt_Salience_Gaps_Table <- NaturalCourt_Salience_Gaps %>%
  dplyr::transmute(

    Period = Court_Period,

    Chief,

    Band = Series,

    `Reference cases` = N_Cases_Reference,

    `Comparison cases` = N_Cases_Comparison,

    `Mean |phi| gap` = round(Phi_Gap, 3),

    `Normalized |phi| gap` = round(Phi_Normalized_Gap, 3),

    `Strict-split gap` = scales::percent(Strict_Split_Gap, accuracy = 0.1)
)

# -----
# Reshape raw and normalized phi gaps for plotting
# -----

NaturalCourt_Salience_Gaps_Long <- NaturalCourt_Salience_Gaps %>%
  dplyr::select(
    naturalCourt, Court_Index, Court_Period, Series, N_Cases_Reference,
    N_Cases_Comparison, Eligible_For_Plot, Phi_Gap, Phi_Normalized_Gap) %>%

```

```

tidyr::pivot_longer(
  cols = c(Phi_Gap, Phi_Normalized_Gap), names_to = "Gap_Measure",
  values_to = "Gap_Value") %>% dplyr::mutate(

  Gap_Measure = dplyr::recode(
    Gap_Measure, Phi_Gap = "Mean absolute phi gap", Phi_Normalized_Gap =
      "Mean margin-normalized phi gap"),

  Gap_Measure = factor(
    Gap_Measure, levels = c(
      "Mean absolute phi gap", "Mean margin-normalized phi gap")),

  Series =
    factor(Series, levels = c("CSI 2-4", "CSI 5-6", "CSI >= 7")),

  Gap_Value_Plot = dplyr::if_else(
    Eligible_For_Plot & !is.na(Gap_Value), Gap_Value, NA_real_) %>%
dplyr::group_by(Gap_Measure, Series) %>%
dplyr::arrange(Court_Index, .by_group = TRUE) %>% dplyr::mutate(

  Gap_Available = !is.na(Gap_Value_Plot),

  Plot_Run = cumsum(
    Gap_Available != dplyr::lag(
      Gap_Available, default = dplyr::first(Gap_Available)))) %>%
dplyr::ungroup()

# -----
# Natural Court axis labels
# -----

NaturalCourt_Axis <- NaturalCourt_Descriptives %>%
  dplyr::arrange(Court_Index) %>%
  dplyr::distinct(Court_Index, Court_Period)

# -----
# Colors
# -----

natural_court_gap_colors <- c(
  "CSI 2-4" = "#B8C95A", "CSI 5-6" = "#E9962F", "CSI >= 7" = "#9E0142")

```

```

# -----
# Natural Court salience-gap figure
# -----

p_natural_court_gaps <- ggplot2::ggplot() +

  ggplot2::geom_hline(
    yintercept = 0, color = "grey55", linewidth = 0.30) +

  ggplot2::geom_line(
    data = NaturalCourt_Salience_Gaps_Long, ggplot2::aes(
      x = Court_Index, y = Gap_Value_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

  ggplot2::geom_line(
    data = NaturalCourt_Salience_Gaps_Long, ggplot2::aes(
      x = Court_Index, y = Gap_Value_Plot, group =
        interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

  ggplot2::geom_point(
    data = NaturalCourt_Salience_Gaps_Long,
    ggplot2::aes(x = Court_Index, y = Gap_Value_Plot, color = Series),
    size = 1.5, alpha = 0.97, na.rm = TRUE) +

  ggplot2::facet_wrap(~ Gap_Measure, ncol = 1) +

  ggplot2::scale_color_manual(
    values = natural_court_gap_colors,
    breaks = c("CSI 2-4", "CSI 5-6", "CSI >= 7"), name = NULL) +

  ggplot2::scale_x_continuous(
    breaks = NaturalCourt_Axis$Court_Index, labels =
      NaturalCourt_Axis$Court_Period, expand =
        ggplot2::expansion(mult = c(0.01, 0.01))) +

  ggplot2::labs(
    title = "Salience gaps across Supreme Court compositions", x = NULL, y =
      "Difference from CSI < 2 in phi coefficient") +

  ggplot2::theme_minimal() +

```

```

ggplot2::theme(

  legend.position = "bottom",

  legend.direction = "horizontal",

  panel.grid.minor = ggplot2::element_blank(),

  panel.grid.major.x = ggplot2::element_blank(),

  panel.spacing = grid::unit(0.8, "lines"),

  strip.text = ggplot2::element_text(face = "bold", hjust = 0.5),

  plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),

  axis.text.x =
    ggplot2::element_text(angle = 45, hjust = 1, vjust = 1, size = 7),

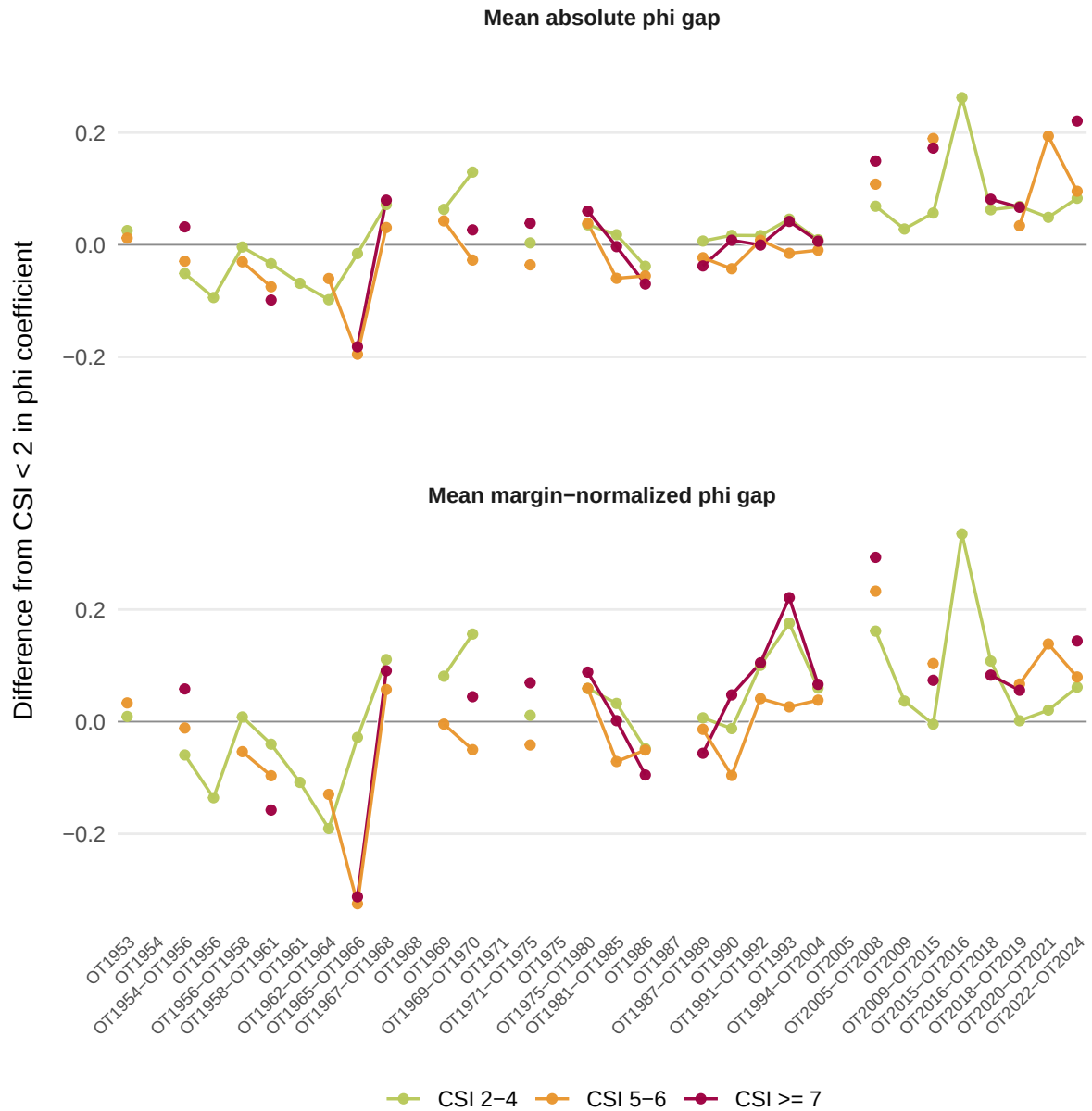
  axis.title.y =
    ggplot2::element_text(margin = ggplot2::margin(r = 10)),

  legend.margin = ggplot2::margin(t = 5))

print(p_natural_court_gaps)

```

Saliency gaps across Supreme Court compositions



```
ggplot2::ggsave(
  filename = file.path(
    fig_dir, "03_phi_saliency_gap_by_natural_court.emf"), plot =
    p_natural_court_gaps, device = devEMF::emf, width = 6.27, height = 6.8,
    units = "in")
```

Strict Appointing-Party Split Proportions

Purpose

The following code provides an intuitive binary complement to the continuous phi measures by showing the proportion of salient contested cases producing a **strict appointing-party split**.

A strict split occurs when all participating Republican-appointed justices fall on one disposition side and all participating Democratic-appointed justices fall on the other.

Unlike the salience-gap figures, this analysis retains the principal **nested CSI thresholds from 3 through 7**. The question here is not how one salience band differs from a common low-salience reference, but rather:

Among contested cases at least this salient, what proportion produce a complete appointing-party split?

The rolling statistic uses the same trailing windows as the other historical figures. Estimates based on fewer than five contested cases are suppressed for plotting only.

A diagnostic observation is retained for the rolling window ending in OT2024. That window contains OT2020–OT2024 and should reproduce the previously observed **20 strict splits among 32 CSI ≥ 5 contested cases, or 62.5%**, if the final merged data reproduce the exploratory result.

Created objects

- **Rolling_Strict_Thresholds** — plotting dataset containing trailing strict-split proportions for CSI thresholds 3–7.
- **p_strict_thresholds** — unshaded rolling strict-split figure.
- **p_strict_thresholds_shaded** — corresponding figure with Natural Court composition shading.
- **Modern_Strict_Split_Check** — diagnostic observation for CSI ≥ 5 during OT2020–OT2024.
- **Modern_Strict_Split_Check_Table** — reader-facing version of that diagnostic.

```

# --- Strict appointing-party split proportions -----
# -----
# Minimum sample size for displayed rolling estimates
# -----

min_cases_strict_plot <- 5L

# -----
# Principal CSI thresholds
# -----

strict_series_levels <- paste0("CSI >= ", csi_cutoffs)

# -----
# Prepare rolling strict-split plotting data
# -----

Rolling_Strict_Thresholds <- Rolling_Phi_Thresholds %>%
  dplyr::filter(CSI_Cutoff %in% csi_cutoffs) %>% dplyr::mutate(

    Series = paste0("CSI >= ", CSI_Cutoff),

    Series = factor(Series, levels = strict_series_levels),

    Eligible_For_Plot = N_Cases >= min_cases_strict_plot &
      !is.na(Strict_Split_Rate),

    Strict_Split_Rate_Plot =
      dplyr::if_else(Eligible_For_Plot, Strict_Split_Rate, NA_real_) %>%
dplyr::group_by(Series) %>% dplyr::arrange(Year, .by_group = TRUE) %>%
dplyr::mutate(

  Plot_Available = !is.na(Strict_Split_Rate_Plot),

  Plot_Run = cumsum(
    Plot_Available != dplyr::lag(
      Plot_Available, default = dplyr::first(Plot_Available)))
) %>% dplyr::ungroup()

# -----
# Strict-split colors

```

```

# -----

strict_plot_colors <- c(
  "CSI >= 3" = "#B8C95A", "CSI >= 4" = "#D9B43B", "CSI >= 5" = "#E9962F",
  "CSI >= 6" = "#E85B3A", "CSI >= 7" = "#9E0142")

# -----
# Shared scales and theme
# -----

strict_split_scales <- list(

  ggplot2::scale_color_manual(
    values = strict_plot_colors, breaks = strict_series_levels, labels =
      strict_series_levels, name = NULL, drop = FALSE),

  ggplot2::guides(
    color = ggplot2::guide_legend(
      nrow = 1, byrow = TRUE, override.aes =
        list(linewidth = 0.62, alpha = 1))),

  ggplot2::scale_x_continuous(
    limits = c(analysis_start_term, analysis_end_term),
    breaks = seq(1960, 2020, by = 10), expand =
      ggplot2::expansion(mult = c(0, 0))),

  ggplot2::scale_y_continuous(
    limits = c(0, 1), breaks = seq(0, 1, by = 0.2), labels =
      scales::percent_format(accuracy = 1), expand =
      ggplot2::expansion(mult = c(0, 0.02))),

  ggplot2::labs(
    title = "Strict appointing-party splits in salient contested cases", x =
      "Year", y = "Strict appointing-party split rate"),

  ggplot2::theme_minimal(),

  ggplot2::theme(

    legend.position = "bottom",

    legend.direction = "horizontal",

```

```

panel.grid.minor = ggplot2::element_blank(),

panel.grid.major.x = ggplot2::element_blank(),

plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),

axis.title.x =
  ggplot2::element_text(margin = ggplot2::margin(t = 10)),

axis.title.y =
  ggplot2::element_text(margin = ggplot2::margin(r = 10)),

legend.margin = ggplot2::margin(t = 5)))

# -----
# A. Figure without Natural Court shading
# -----

p_strict_thresholds <- ggplot2::ggplot() +

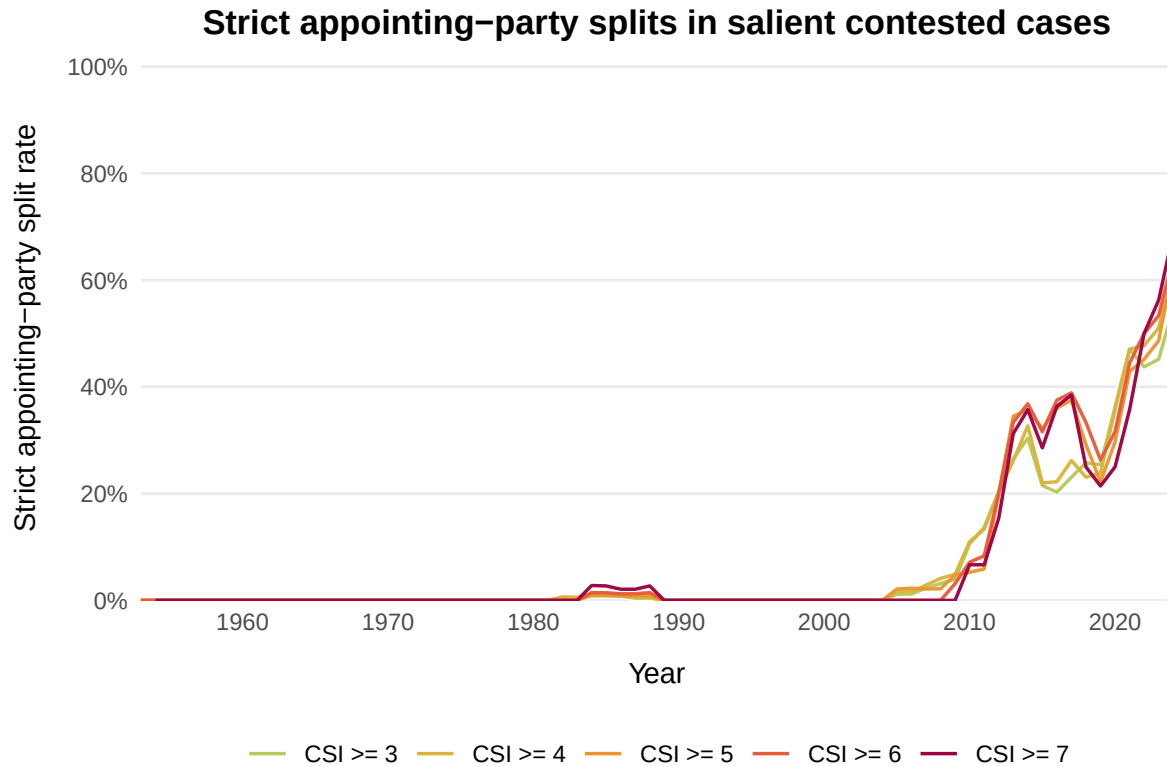
# White underlay.
ggplot2::geom_line(
  data = Rolling_Strict_Thresholds, ggplot2::aes(
    x = Year, y = Strict_Split_Rate_Plot, group =
      interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

# Strict-split trajectories.
ggplot2::geom_line(
  data = Rolling_Strict_Thresholds, ggplot2::aes(
    x = Year, y = Strict_Split_Rate_Plot, group =
      interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

strict_split_scales

print(p_strict_thresholds)

```



```
ggplot2::ggsave(
  filename = file.path(
    fig_dir, "04_strict_party_split_rolling_thresholds.emf"), plot =
    p_strict_thresholds, device = devEMF::emf, width = 6.27, height = 4.2,
    units = "in")

# -----
# B. Figure with Natural Court composition shading
# -----

p_strict_thresholds_shaded <- ggplot2::ggplot() +

  ggplot2::geom_rect(
    data = NaturalCourt_Shading, ggplot2::aes(
      xmin = xmin, xmax = xmax, ymin = -Inf, ymax = Inf, fill =
        Shade_Colour, alpha = Shade_Intensity), inherit.aes = FALSE) +

  ggplot2::scale_fill_identity(guide = "none") +
```

```

ggplot2::scale_alpha_identity(guide = "none") +

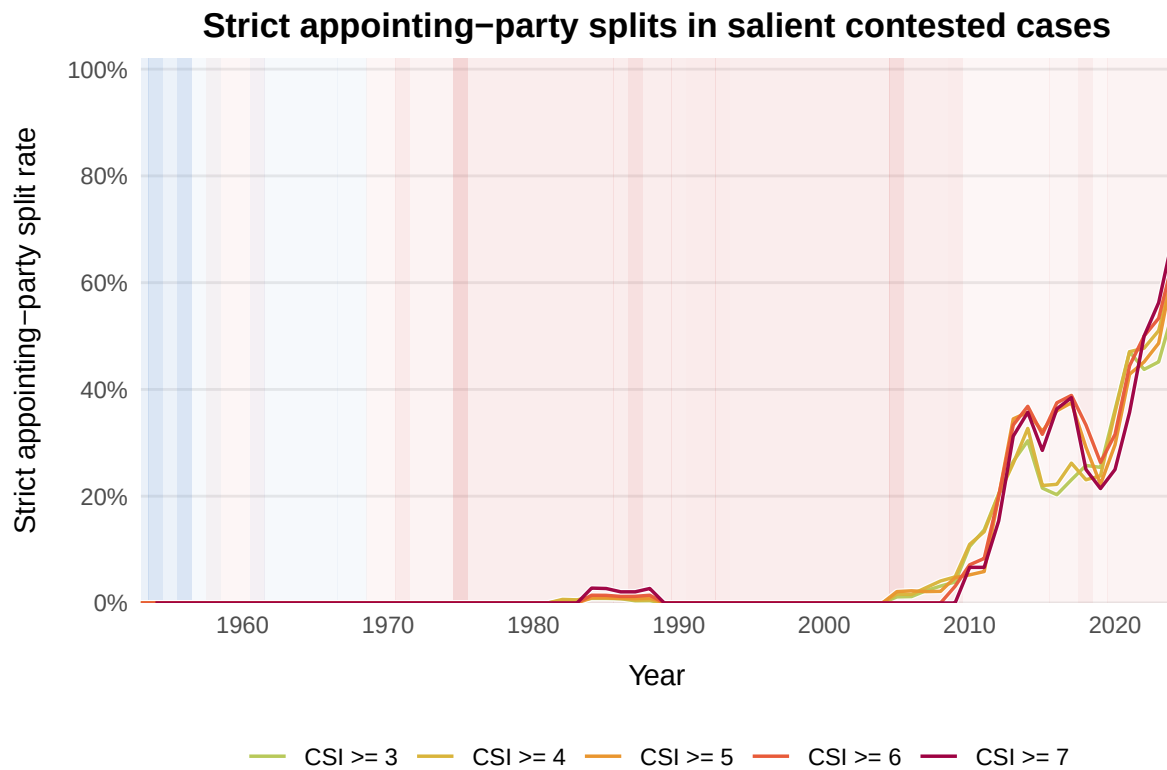
ggplot2::geom_line(
  data = Rolling_Strict_Thresholds, ggplot2::aes(
    x = Year, y = Strict_Split_Rate_Plot, group =
      interaction(Series, Plot_Run, drop = TRUE)), color = "white",
    linewidth = 1.05, alpha = 0.90, show.legend = FALSE, na.rm = TRUE) +

ggplot2::geom_line(
  data = Rolling_Strict_Thresholds, ggplot2::aes(
    x = Year, y = Strict_Split_Rate_Plot, group =
      interaction(Series, Plot_Run, drop = TRUE), color = Series),
    linewidth = 0.62, alpha = 0.97, na.rm = TRUE) +

strict_split_scales

print(p_strict_thresholds_shaded)

```



```

ggplot2::ggsave(
  filename = file.path(
    fig_dir, "04_strict_party_split_rolling_thresholds_shaded.emf"), plot =
    p_strict_thresholds_shaded, device = devEMF::emf, width = 6.27, height =
    4.2, units = "in")

# -----
# Diagnostic check corresponding to OT2020-OT2024
#
# Under the current trailing-window definition, Year = 2024 represents:
#
# OT2020, OT2021, OT2022, OT2023, OT2024.
#
# This replaces the earlier centered-window check at Center_Term = 2022.
# -----

Modern_Strict_Split_Check <- Rolling_Phi_Thresholds %>%
  dplyr::filter(Year == 2024, CSI_Cutoff == 5) %>% dplyr::select(
    Year, Window_Start, Window_End, CSI_Cutoff, N_Cases, Strict_Split_N,
    Strict_Split_Rate)

# -----
# Reader-facing diagnostic
# -----

Modern_Strict_Split_Check_Table <- Modern_Strict_Split_Check %>%
  dplyr::transmute(

    Window = paste0("OT", Window_Start, "-OT", Window_End),

    `CSI threshold` = paste0("CSI >= ", CSI_Cutoff),

    Cases = N_Cases,

    `Strict splits` = Strict_Split_N,

    `Strict-split rate` =
      scales::percent(Strict_Split_Rate, accuracy = 0.1))

knitr::kable(
  Modern_Strict_Split_Check_Table, caption =
    paste0(

```

```

    "Diagnostic strict-split rate for CSI >= 5 contested cases ",
    "during OT2020-OT2024"
),
booktabs = TRUE, align = c("l", "l", "r", "r", "r"))

```

Table 10: Diagnostic strict-split rate for CSI ≥ 5 contested cases during OT2020-OT2024

Window	CSI threshold	Cases	Strict splits	Strict-split rate
OT2020-OT2024	CSI ≥ 5	32	20	62.5%

Predictive Analysis of the Saliency–Alignment Relationship

Vote Division by Case Saliency

Purpose

The following code examines whether case saliency is associated with the **propensity for substantial disagreement in the first place**, before restricting the analysis to contested cases.

Unlike the subsequent phi analyses, this section uses the complete merged CSI case universe, `SCOTUS_CaseData`. Cases are classified according to whether the decision was unanimous, contained exactly one minority vote, contained at least two minority votes, or was closely divided with at least three minority votes.

To remain consistent with the descriptive saliency-gap analysis, cases are grouped into **CSI < 2**, **CSI 2–4**, **CSI 5–6**, and **CSI ≥ 7**. The contested and closely divided categories are intentionally nested: closely divided cases form a subset of cases with at least two minority votes.

This distinguishes two outcomes: One, whether saliency is associated with substantial disagreement, and two, among contested cases, whether that disagreement is aligned by appointing-president party.

Created objects

- **Vote_Division_Data** — complete CSI case universe with saliency-band and vote-division indicators.
- **Vote_Division_By_Saliency** — descriptive vote-division rates by saliency band.
- **Vote_Division_Table** — publication-facing table of vote-division patterns.

```
# --- Vote division by case saliency -----  
  
# -----  
# Saliency bands  
# -----  
  
vote_division_saliency_levels <- c(  
  "CSI < 2", "CSI 2-4", "CSI 5-6", "CSI ≥ 7")  
  
# -----  
# Full case universe
```

```

# -----
Vote_Division_Data <- SCOTUS_CaseData %>% dplyr::filter(
  term >= analysis_start_term, term <= analysis_end_term, !is.na(csi),
  !is.na(minVotes)) %>% dplyr::mutate(

  Salience_Band = dplyr::case_when(
    csi < 2 ~ "CSI < 2",
    csi >= 2 & csi < 5 ~ "CSI 2-4",
    csi >= 5 & csi < 7 ~ "CSI 5-6",
    csi >= 7 ~ "CSI >= 7",
    TRUE ~ NA_character_),

  Salience_Band =
    factor(Salience_Band, levels = vote_division_salience_levels),

  Unanimous = minVotes == 0,

  One_Minority_Vote = minVotes == 1,

  Contested = minVotes >= 2,

  Closely_Divided = minVotes >= 3) %>%
dplyr::filter(!is.na(Salience_Band))

# -----
# Descriptive vote-division statistics
# -----

Vote_Division_By_Salience <- Vote_Division_Data %>%
  dplyr::group_by(Salience_Band) %>% dplyr::summarise(

  N_Cases = dplyr::n(),

  Unanimous_Rate = safe_rate(Unanimous),

  One_Minority_Rate = safe_rate(One_Minority_Vote),

  Contested_Rate = safe_rate(Contested),

  Closely_Divided_Rate = safe_rate(Closely_Divided),

```

```

.groups = "drop")

# -----
# Publication-facing table
# -----

Vote_Division_Table <- Vote_Division_By_Salience %>% dplyr::transmute(

  `CSI band` = as.character(Salience_Band),

  Cases = N_Cases,

  Unanimous = scales::percent(Unanimous_Rate, accuracy = 0.1),

  `One minority vote` =
    scales::percent(One_Minority_Rate, accuracy = 0.1),

  `Contested (2+ minority votes)` =
    scales::percent(Contested_Rate, accuracy = 0.1),

  `Closely divided (3+ minority votes)` =
    scales::percent(Closely_Divided_Rate, accuracy = 0.1))

knitr::kable(
  Vote_Division_Table, caption = "Vote division by case salience",
  booktabs = TRUE, align = c("l", "r", "r", "r", "r", "r"))

```

Table 11: Vote division by case salience

CSI band	Cases	Unanimous	One minority vote	Contested (2+ minority votes)	Closely divided (3+ minority votes)
CSI < 2	2932	47.3%	11.3%	41.3%	27.6%
CSI 2-4	3171	36.7%	11.6%	51.7%	36.5%
CSI 5-6	992	25.6%	11.1%	63.3%	46.8%
CSI >= 7	505	22.2%	8.9%	68.9%	54.3%

CSI Regression Model

Purpose

Estimate whether case salience is associated with the probability that a Supreme Court case is contested. A contested case is defined as a case with at least two minority votes.

The model uses CSI as a continuous predictor, includes October Term fixed effects, and clusters standard errors by October Term.

Created objects

- **Contested_Model_Data** — data used in the CSI regression model, including the binary contested-case indicator.
- **Contested_CSI_Model** — regression of contested-case status on CSI with October Term fixed effects.
- **Contested_CSI_VCov** — term-clustered variance-covariance matrix.
- **Contested_CSI_Result** — regression results with clustered standard errors.
- **Contested_CSI_Estimate** — estimated CSI coefficient.
- **Contested_CSI_SE** — clustered standard error for the CSI coefficient.
- **Contested_CSI_P** — p-value for the CSI coefficient.
- **Contested_CSI_Lower** and **Contested_CSI_Upper** — lower and upper bounds of the 95% confidence interval.
- **Contested_CSI_Table** — publication-facing regression table.

```
# --- CSI regression models -----  
  
# -----  
# Regression dataset  
# -----  
  
contested_csi_period_levels <- c(  
  "1955-1964",  
  "1965-1974",  
  "1975-1984",  
  "1985-1994",  
  "1995-2004",  
  "2005-2009",
```

```

"2010-2014",
"2015-2019",
"2020-2024"
)

Contested_CSI_Data <- SCOTUS_CaseData %>%
  dplyr::filter(
    term >= analysis_start_term,
    term <= analysis_end_term,
    !is.na(csi),
    !is.na(minVotes)
  ) %>%
  dplyr::mutate(

    contested_case = as.integer(minVotes >= 2),

    Regression_Period = dplyr::case_when(
      term >= 1955 & term <= 1964 ~ "1955-1964",
      term >= 1965 & term <= 1974 ~ "1965-1974",
      term >= 1975 & term <= 1984 ~ "1975-1984",
      term >= 1985 & term <= 1994 ~ "1985-1994",
      term >= 1995 & term <= 2004 ~ "1995-2004",
      term >= 2005 & term <= 2009 ~ "2005-2009",
      term >= 2010 & term <= 2014 ~ "2010-2014",
      term >= 2015 & term <= 2019 ~ "2015-2019",
      term >= 2020 & term <= 2024 ~ "2020-2024",
      TRUE ~ NA_character_
    ),

    Regression_Period = factor(
      Regression_Period,
      levels = contested_csi_period_levels
    )
  )

# -----
# Clustered-result helper
# -----

extract_contested_csi <- function(
  model, vcov_matrix, data, period) {

```

```

estimate <- stats::coef(model)[["csi"]]

standard_error <- unname(
  sqrt(vcov_matrix["csi", "csi"])
)

n_terms <- dplyr::n_distinct(data$term)

cluster_df <- n_terms - 1L

critical_value <- stats::qt(
  0.975,
  df = cluster_df
)

t_value <- estimate / standard_error

p_value <- 2 * stats::pt(
  abs(t_value),
  df = cluster_df,
  lower.tail = FALSE
)

tibble::tibble(

  Period = period,

  Estimate = as.numeric(estimate),

  Clustered_SE = as.numeric(standard_error),

  CI_Lower = as.numeric(
    estimate - critical_value * standard_error
  ),

  CI_Upper = as.numeric(
    estimate + critical_value * standard_error
  ),

  P_Value = as.numeric(p_value),

  N_Cases = as.integer(stats::nobs(model)),

```

```

    N_Terms = as.integer(n_terms)
  )
}

# -----
# A. Overall OT1953-OT2024 regression
# -----

m_contested_csi_all <- stats::lm(
  contested_case ~ csi + factor(term),
  data = Contested_CSI_Data
)

vcov_contested_csi_all <- sandwich::vcovCL(
  m_contested_csi_all,
  cluster = Contested_CSI_Data$term,
  type = "HC1"
)

Contested_CSI_Results_Overall <- extract_contested_csi(
  model = m_contested_csi_all,
  vcov_matrix = vcov_contested_csi_all,
  data = Contested_CSI_Data,
  period = "OT1953-OT2024"
)

# -----
# B. Separate regressions by historical period
# -----

Contested_CSI_Period_Data <- Contested_CSI_Data %>%
  dplyr::filter(
    !is.na(Regression_Period)
  ) %>%
  split(
    .$Regression_Period
  )

m_contested_csi_periods <- purrr::map(
  Contested_CSI_Period_Data,
  function(data_i) {

```

```

    stats::lm(
      contested_case ~ csi + factor(term),
      data = data_i
    )
  }
)

vcov_contested_csi_periods <- purrr::map2(
  m_contested_csi_periods,
  Contested_CSI_Period_Data,
  function(model_i, data_i) {

    sandwich::vcovCL(
      model_i,
      cluster = data_i$term,
      type = "HC1"
    )
  }
)

Contested_CSI_Results_By_Period <- purrr::map_dfr(
  names(m_contested_csi_periods),
  function(period_i) {

    extract_contested_csi(
      model = m_contested_csi_periods[[period_i]],
      vcov_matrix =
        vcov_contested_csi_periods[[period_i]],
      data = Contested_CSI_Period_Data[[period_i]],
      period = period_i
    )
  }
)

# -----
# Combine overall and period-specific results
# -----

Contested_CSI_Results <- dplyr::bind_rows(
  Contested_CSI_Results_Overall,
  Contested_CSI_Results_By_Period
)

```

```

# -----
# Publication-facing regression table
# -----

Contested_CSI_Table <- Contested_CSI_Results %>%
  dplyr::mutate(

    `95% CI` = sprintf(
      "%.3f to %.3f",
      CI_Lower,
      CI_Upper
    ),

    `p value` = dplyr::if_else(
      P_Value < 0.001,
      "<0.001",
      sprintf("%.3f", P_Value)
    )
  ) %>%
  dplyr::transmute(

    Period,

    Estimate = round(Estimate, 3),

    `Clustered SE` = round(Clustered_SE, 3),

    `95% CI`,

    `p value`,

    Cases = N_Cases,

    Terms = N_Terms
  )

knitr::kable(
  Contested_CSI_Table,
  caption =
    "Case salience and the probability of a contested Supreme Court decision",
  booktabs = TRUE,
  align = c("l", "r", "r", "l", "r", "r", "r")
)

```

)

Table 12: Case salience and the probability of a contested Supreme Court decision

Period	Estimate	Clustered SE	95% CI	p value	Cases	Terms
OT1953-OT2024	0.044	0.003	0.037 to 0.050	<0.001	7600	72
1955-1964	0.017	0.010	-0.006 to 0.040	0.134	1182	10
1965-1974	0.033	0.009	0.014 to 0.052	0.004	1287	10
1975-1984	0.042	0.005	0.030 to 0.053	<0.001	1418	10
1985-1994	0.047	0.006	0.033 to 0.060	<0.001	1311	10
1995-2004	0.065	0.009	0.046 to 0.084	<0.001	878	10
2005-2009	0.033	0.011	0.003 to 0.062	0.036	367	5
2010-2014	0.065	0.009	0.040 to 0.091	0.002	358	5
2015-2019	0.066	0.007	0.048 to 0.085	<0.001	319	5
2020-2024	0.054	0.010	0.027 to 0.080	0.005	301	5

Breakpoint Diagnostics

Purpose

The following code evaluates how sharply the previously estimated breakpoint in the salience–alignment relationship is identified.

First, the candidate breakpoint range is widened from OT1990–OT2018 to **OT1980–OT2018**. The wider range checks whether the earlier normalized-phi interval was constrained by the OT1990 lower bound.

The code then plots the complete residual-sum-of-squares profile across candidate terms. A narrow RSS minimum indicates a more precisely located breakpoint; a flatter profile indicates greater timing uncertainty.

A second term-cluster bootstrap is conducted using the wider candidate range. The frequency with which each candidate term is selected is displayed directly. The diagnostics show whether the breakpoint is concentrated around a narrow group of terms or is weakly identified.

Created objects

- **break_candidates_wide** — widened OT1980–OT2018 breakpoint-search range.
- **Breakpoint_Search_Raw_Wide** — widened raw-phi breakpoint profile.

- **Breakpoint_Search_Normalized_Wide** — widened normalized-phi breakpoint profile.
- **Breakpoint_Best_Wide** — best-fitting widened breakpoint for each outcome.
- **Breakpoint_Bootstrap_Wide** — term-cluster bootstrap estimates using the widened search range.
- **Breakpoint_Bootstrap_Frequency** — frequency with which each candidate term is selected.
- **Breakpoint_Diagnostic_Summary** — publication-facing comparison of original and widened breakpoint estimates.
- **p_breakpoint_profile** — complete RSS-profile diagnostic.
- **p_breakpoint_bootstrap** — bootstrap breakpoint-frequency diagnostic.

```
# --- Breakpoint diagnostics -----
# -----
# Wider candidate range
#
# The original search used OT1990-OT2018.
#
# The widened search tests whether the normalized result was being constrained
# by the lower boundary of that range.
# -----

break_candidates_wide <- seq.int(1980L, 2018L)

# -----
# Wider point-estimate searches
# -----

Breakpoint_Search_Raw_Wide <- search_breakpoint(
  Breakpoint_Data_Raw, break_candidates_wide)

Breakpoint_Search_Normalized_Wide <- search_breakpoint(
  Breakpoint_Data_Normalized, break_candidates_wide)

# -----
# Best-fitting widened breakpoint
# -----
```

```

Breakpoint_Best_Raw_Wide <- Breakpoint_Search_Raw_Wide %>%
  dplyr::filter(is.finite(RSS)) %>%
  dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1)

Breakpoint_Best_Normalized_Wide <- Breakpoint_Search_Normalized_Wide %>%
  dplyr::filter(is.finite(RSS)) %>%
  dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1)

Breakpoint_Best_Wide <- dplyr::bind_rows(

  Breakpoint_Best_Raw_Wide %>%
    dplyr::mutate(Measure = "Mean absolute phi"),

  Breakpoint_Best_Normalized_Wide %>%
    dplyr::mutate(Measure = "Margin-normalized phi"))

# -----
# Complete RSS profiles
# -----

Breakpoint_Profile <- dplyr::bind_rows(

  Breakpoint_Search_Raw_Wide %>%
    dplyr::mutate(Measure = "Mean absolute phi"),

  Breakpoint_Search_Normalized_Wide %>%
    dplyr::mutate(Measure = "Margin-normalized phi")) %>% dplyr::mutate(

  Measure = factor(
    Measure, levels = c("Mean absolute phi", "Margin-normalized phi")
  ))

# -----
# RSS-profile figure
# -----

p_breakpoint_profile <- ggplot2::ggplot(
  Breakpoint_Profile, ggplot2::aes(x = Break_Term, y = Delta_RSS)) +

  ggplot2::geom_line(linewidth = 0.70) +

  ggplot2::geom_point(size = 1.4) +

```

```

ggplot2::geom_vline(
  data = Breakpoint_Best_Wide, ggplot2::aes(xintercept = Break_Term),
  linetype = "dashed", linewidth = 0.45) +

ggplot2::facet_wrap(~ Measure, ncol = 1, scales = "free_y") +

ggplot2::scale_x_continuous(breaks = seq(1980, 2015, by = 5)) +

ggplot2::labs(
  title = "Breakpoint fit across candidate Supreme Court terms", x =
    "Candidate breakpoint term", y = "Difference from minimum RSS") +

ggplot2::theme_minimal() +

ggplot2::theme(

  panel.grid.minor = ggplot2::element_blank(),

  panel.grid.major.x = ggplot2::element_blank(),

  panel.spacing = grid::unit(0.8, "lines"),

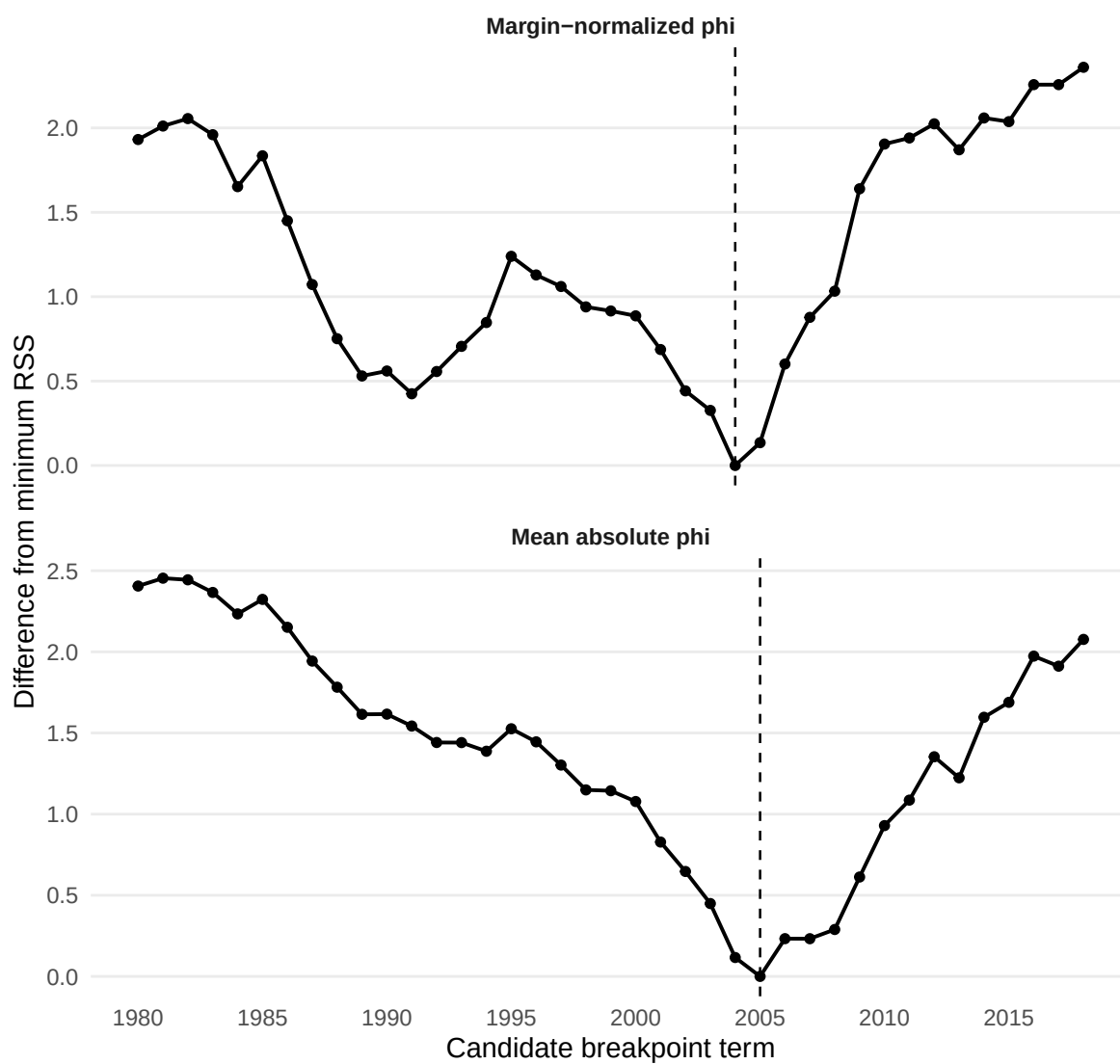
  strip.text = ggplot2::element_text(face = "bold", hjust = 0.5),

  plot.title = ggplot2::element_text(face = "bold", hjust = 0.5))

print(p_breakpoint_profile)

```

Breakpoint fit across candidate Supreme Court terms



```
ggplot2::ggsave(
  filename = file.path(fig_dir, "05_breakpoint_rss_profile.emf"), plot =
    p_breakpoint_profile, device = devEMF::emf, width = 6.27, height = 6.2,
  units = "in")
```

```
# -----
# Wider term-cluster bootstrap
# -----
```

```

break_diag_boot_reps <- 1000L

set.seed(20260908)

Breakpoint_Bootstrap_Wide <- purrr::map_dfr(
  seq_len(break_diag_boot_reps), function(r) {

    sampled_terms <- sample(
      common_break_terms, size = length(common_break_terms), replace =
        TRUE)

    term_frequency <- table(
      factor(sampled_terms, levels = common_break_terms))

    term_weights <- stats::setNames(
      as.numeric(term_frequency), as.character(common_break_terms))

    raw_weights <- unname(
      term_weights[
        as.character(Breakpoint_Data_Raw$term)])

    normalized_weights <- unname(
      term_weights[
        as.character(Breakpoint_Data_Normalized$term)])

    raw_search <- search_breakpoint(
      Breakpoint_Data_Raw, break_candidates_wide, weights = raw_weights)

    normalized_search <- search_breakpoint(
      Breakpoint_Data_Normalized, break_candidates_wide, weights =
        normalized_weights)

    raw_break <- raw_search %>% dplyr::filter(is.finite(RSS)) %>%
      dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1) %>%
      dplyr::pull(Break_Term)

    normalized_break <- normalized_search %>%
      dplyr::filter(is.finite(RSS)) %>%
      dplyr::arrange(RSS, Break_Term) %>% dplyr::slice(1) %>%
      dplyr::pull(Break_Term)

    tibble::tibble(

```

```

    Replicate = r,

    Raw_Break = raw_break,

    Normalized_Break = normalized_break)
})

# -----
# Bootstrap selection frequencies
# -----

Breakpoint_Bootstrap_Frequency <- dplyr::bind_rows(

  Breakpoint_Bootstrap_Wide %>%
    dplyr::count(Raw_Break, name = "Selections") %>%
    dplyr::rename(Break_Term = Raw_Break) %>%
    dplyr::mutate(Measure = "Mean absolute phi"),

  Breakpoint_Bootstrap_Wide %>%
    dplyr::count(Normalized_Break, name = "Selections") %>%
    dplyr::rename(Break_Term = Normalized_Break) %>%
    dplyr::mutate(Measure = "Margin-normalized phi")) %>%
dplyr::group_by(Measure) %>% tidyr::complete(
  Break_Term = break_candidates_wide, fill = list(Selections = 0L)) %>%
dplyr::mutate(

  Selection_Rate = Selections / sum(Selections),

  Measure = factor(
    Measure, levels = c("Mean absolute phi", "Margin-normalized phi")
)) %>% dplyr::ungroup()

# -----
# Bootstrap-frequency figure
# -----

p_breakpoint_bootstrap <- ggplot2::ggplot(
  Breakpoint_Bootstrap_Frequency,
  ggplot2::aes(x = Break_Term, y = Selection_Rate)) +

  ggplot2::geom_line(linewidth = 0.65) +

```

```

ggplot2::geom_point(size = 1.3) +

ggplot2::facet_wrap(~ Measure, ncol = 1) +

ggplot2::scale_x_continuous(breaks = seq(1980, 2015, by = 5)) +

ggplot2::scale_y_continuous(
  labels = scales::percent_format(accuracy = 1)) +

ggplot2::labs(
  title = "Bootstrap distribution of estimated breakpoint terms", x =
    "Estimated breakpoint term", y = "Share of bootstrap samples") +

ggplot2::theme_minimal() +

ggplot2::theme(

  panel.grid.minor = ggplot2::element_blank(),

  panel.grid.major.x = ggplot2::element_blank(),

  panel.spacing = grid::unit(0.8, "lines"),

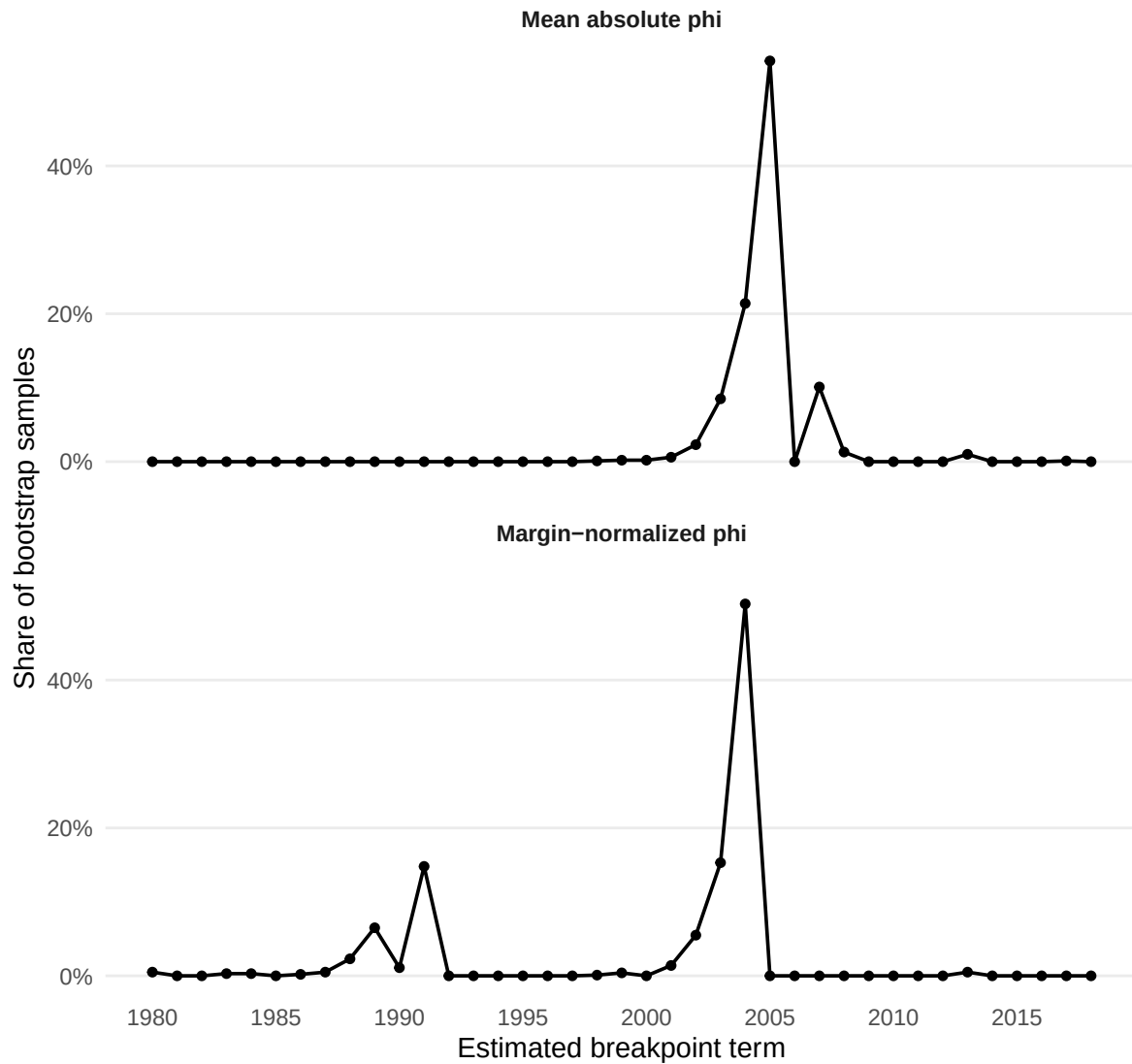
  strip.text = ggplot2::element_text(face = "bold", hjust = 0.5),

  plot.title = ggplot2::element_text(face = "bold", hjust = 0.5))

print(p_breakpoint_bootstrap)

```

Bootstrap distribution of estimated breakpoint terms



```
ggplot2::ggsave(  
  filename = file.path(  
    fig_dir, "06_breakpoint_bootstrap_distribution.emf"), plot =  
    p_breakpoint_bootstrap, device = devEMF::emf, width = 6.27, height =  
    6.2, units = "in")
```

```
# -----  
# Diagnostic summary  
# -----
```

```

raw_wide_interval <- breakpoint_interval(
  Breakpoint_Bootstrap_Wide$Raw_Break)

normalized_wide_interval <- breakpoint_interval(
  Breakpoint_Bootstrap_Wide$Normalized_Break)

Breakpoint_Diagnostic_Summary <- tibble::tibble(

  Measure = c("Mean absolute phi", "Margin-normalized phi"),

  Original_Break = c(
    Breakpoint_Best_Raw$Break_Term, Breakpoint_Best_Normalized$Break_Term),

  Wide_Break = c(
    Breakpoint_Best_Raw_Wide$Break_Term,
    Breakpoint_Best_Normalized_Wide$Break_Term),

  Bootstrap_Median = c(
    raw_wide_interval[
      "Median"], normalized_wide_interval[
      "Median"]),

  CI_Lower = c(
    raw_wide_interval[
      "Lower"], normalized_wide_interval[
      "Lower"]),

  CI_Upper = c(
    raw_wide_interval[
      "Upper"], normalized_wide_interval[
      "Upper"]))

Breakpoint_Diagnostic_Table <- Breakpoint_Diagnostic_Summary %>%
  dplyr::transmute(

    Measure,

    `Original search` = paste0("OT", Original_Break),

    `Wider search` = paste0("OT", Wide_Break),

    `Bootstrap median` = paste0("OT", Bootstrap_Median),

```

```

`95% bootstrap interval` = paste0("OT", CI_Lower, "-OT", CI_Upper))

knitr::kable(
  Breakpoint_Diagnostic_Table, caption =
    "Breakpoint robustness to a wider candidate-term search", booktabs =
    TRUE, align = c("l", "l", "l", "l", "l"))

```

Table 13: Breakpoint robustness to a wider candidate-term search

Measure	Original search	Wider search	Bootstrap median	95% bootstrap interval
Mean absolute phi	OT2005	OT2005	OT2005	OT2002-OT2007
Margin-normalized phi	OT2004	OT2004	OT2004	OT1988-OT2004

Main Regression Models

Purpose

The following code formally tests whether the relationship between continuous case salience and appointing-party alignment has changed over time.

The principal models use the contested-case sample and absorb **Supreme Court term fixed effects**. This is important because it removes term-specific changes in the overall level of appointing-party alignment. The interaction terms therefore test whether more-salient cases are differently aligned than less-salient cases within the same Supreme Court Term, rather than merely detecting that the Court as a whole became more polarized.

Two temporal specifications are estimated. The first allows the CSI slope to change linearly by decade across the full OT1953–OT2024 period. The second uses the theoretically motivated OT2010 division and estimates directly whether the CSI slope differs before and after OT2010.

Raw absolute phi is the principal outcome. Margin-normalized phi is estimated in parallel. Standard errors are clustered by Supreme Court term, and inference uses the number of term clusters rather than treating individual cases as independent temporal observations.

Created objects

- **Model_Data** — contested-case regression sample.
- **m_phi_time** — raw-phi model allowing the CSI slope to change by decade.
- **m_phi_normalized_time** — corresponding normalized-phi model.
- **m_phi_post2010** — raw-phi pre-/post-2010 slope-change model.
- **m_phi_normalized_post2010** — corresponding normalized-phi model.
- **Inferential_Model_Results** — clustered coefficient and slope estimates.
- **Inferential_Model_Table** — publication-facing model summary.

```
# --- Main inferential models -----  
  
# -----  
# Regression dataset  
# -----  
  
Model_Data <- SCOTUS_Contested %>% dplyr::filter(  
  term >= analysis_start_term, term <= analysis_end_term, !is.na(csi),  
  !is.na(phi_abs), !is.na(phi_normalized)) %>% dplyr::mutate(  
    phi_abs = phi_abs * 100,  
    phi_normalized = phi_normalized * 100,  
    phi_abs_scaled = phi_abs / 100,  
    phi_normalized_scaled = phi_normalized / 100
```

```

Post2010 = as.integer(term >= 2010),

Time_Decade = (term - analysis_start_term) / 10)

# -----
# Cluster-robust covariance helper
# -----

cluster_vcov <- function(
  model, cluster) {

  sandwich::vcovCL(model, cluster = cluster, type = "HC1")
}

# -----
# Cluster-robust linear-combination helper
#
# This function allows direct calculation of:
#
#   - pre-period CSI slope;
#   - post-period CSI slope;
#   - change in CSI slope;
#   - change in the CSI slope per decade.
# -----

cluster_contrast <- function(
  model, vcov_matrix, weights, cluster_df, outcome, specification,
  quantity) {

  beta <- stats::coef(model)

  coefficient_names <- names(weights)

  estimate <- sum(weights * beta[coefficient_names])

  variance <- as.numeric(
    t(weights) %*%
    vcov_matrix[
      coefficient_names, coefficient_names, drop = FALSE] %*%
    weights)

  standard_error <- sqrt(variance)

```

```

critical_value <- stats::qt(0.975, df = cluster_df)

t_value <- estimate / standard_error

p_value <- 2 *
  stats::pt(abs(t_value), df = cluster_df, lower.tail = FALSE)

tibble::tibble(

  Outcome = outcome,

  Specification = specification,

  Quantity = quantity,

  Estimate = estimate,

  Clustered_SE = standard_error,

  CI_Lower = estimate - critical_value * standard_error,

  CI_Upper = estimate + critical_value * standard_error,

  P_Value = p_value)
}

# -----
# Number of independent term clusters
# -----

n_term_clusters <- dplyr::n_distinct(Model_Data$term)

cluster_df <- n_term_clusters - 1L

# -----
# A. Continuous historical change in the CSI slope
# -----

m_phi_time <- stats::lm(
  phi_abs ~ csi + csi:Time_Decade + factor(term), data = Model_Data)

vcov_phi_time <- cluster_vcov(m_phi_time, Model_Data$term)

```

```

m_phi_normalized_time <- stats::lm(
  phi_normalized ~ csi + csi:Time_Decade + factor(term), data = Model_Data)

vcov_phi_normalized_time <- cluster_vcov(
  m_phi_normalized_time, Model_Data$term)

# -----
# B. Theory-motivated OT2010 slope change
# -----

m_phi_post2010 <- stats::lm(
  phi_abs ~ csi + csi:Post2010 + factor(term), data = Model_Data)

vcov_phi_post2010 <- cluster_vcov(m_phi_post2010, Model_Data$term)

m_phi_normalized_post2010 <- stats::lm(
  phi_normalized ~ csi + csi:Post2010 + factor(term), data = Model_Data)

vcov_phi_normalized_post2010 <- cluster_vcov(
  m_phi_normalized_post2010, Model_Data$term)

# -----
# Extract quantities of substantive interest
# -----

Inferential_Model_Results <- dplyr::bind_rows(

# -----
# Raw phi: continuous historical change
# -----

  cluster_contrast(
    model = m_phi_time, vcov_matrix = vcov_phi_time, weights =
      c("csi:Time_Decade" = 1), cluster_df = cluster_df, outcome =
        "Mean absolute phi", specification = "Continuous time", quantity =
          "Change in CSI slope per decade"),

# -----
# Normalized phi: continuous historical change
# -----

  cluster_contrast(

```

```

    model = m_phi_normalized_time, vcov_matrix = vcov_phi_normalized_time,
    weights = c("csi:Time_Decade" = 1), cluster_df = cluster_df, outcome =
      "Margin-normalized phi", specification = "Continuous time", quantity =
      "Change in CSI slope per decade"),

# -----
# Raw phi: pre-2010 slope
# -----

cluster_contrast(
  model = m_phi_post2010, vcov_matrix = vcov_phi_post2010, weights =
    c("csi" = 1), cluster_df = cluster_df, outcome =
      "Mean absolute phi", specification = "OT2010 break", quantity =
      "Pre-2010 CSI slope"),

# -----
# Raw phi: post-2010 slope
# -----

cluster_contrast(
  model = m_phi_post2010, vcov_matrix = vcov_phi_post2010, weights =
    c("csi" = 1, "csi:Post2010" = 1), cluster_df = cluster_df, outcome =
      "Mean absolute phi", specification = "OT2010 break", quantity =
      "Post-2010 CSI slope"),

# -----
# Raw phi: slope change
# -----

cluster_contrast(
  model = m_phi_post2010, vcov_matrix = vcov_phi_post2010, weights =
    c("csi:Post2010" = 1), cluster_df = cluster_df, outcome =
      "Mean absolute phi", specification = "OT2010 break", quantity =
      "Change in CSI slope"),

# -----
# Normalized phi: pre-2010 slope
# -----

cluster_contrast(
  model = m_phi_normalized_post2010, vcov_matrix =
    vcov_phi_normalized_post2010, weights = c("csi" = 1), cluster_df =

```

```

    cluster_df, outcome = "Margin-normalized phi", specification =
    "OT2010 break", quantity = "Pre-2010 CSI slope"),

# -----
# Normalized phi: post-2010 slope
# -----

cluster_contrast(
  model = m_phi_normalized_post2010, vcov_matrix =
  vcov_phi_normalized_post2010, weights =
  c("csi" = 1, "csi:Post2010" = 1), cluster_df = cluster_df, outcome =
  "Margin-normalized phi", specification = "OT2010 break", quantity =
  "Post-2010 CSI slope"),

# -----
# Normalized phi: slope change
# -----

cluster_contrast(
  model = m_phi_normalized_post2010, vcov_matrix =
  vcov_phi_normalized_post2010, weights = c("csi:Post2010" = 1),
  cluster_df = cluster_df, outcome = "Margin-normalized phi",
  specification = "OT2010 break", quantity = "Change in CSI slope"))

# -----
# Publication-facing model table
# -----

Inferential_Model_Table <- Inferential_Model_Results %>% dplyr::mutate(

  `95% CI` = paste0(
    sprintf("%.3f", CI_Lower), " to ", sprintf("%.3f", CI_Upper)),

  `p value` = dplyr::if_else(
    P_Value < 0.001, "<0.001", sprintf("%.3f", P_Value))) %>%
dplyr::transmute(

  Outcome,

  Specification,

  Quantity,

```

```

    Estimate = round(Estimate, 3),

    `Clustered SE` = round(Clustered_SE, 3),

    `95% CI`,

    `p value`)

knitr::kable(
  Inferential_Model_Table, caption =
    "Case salience and appointing-party alignment over time", booktabs =
    TRUE, align = c("l", "l", "l", "r", "r", "l", "r"))

```

Table 14: Case salience and appointing-party alignment over time

Outcome	Specification	Quantity	Estimate	Clustered SE	95% CI	p value
Mean absolute phi	Continuous time	Change in CSI slope per decade	0.006	0.001	0.003 to 0.008	<0.001
Margin- normalized phi	Continuous time	Change in CSI slope per decade	0.006	0.001	0.004 to 0.009	<0.001
Mean absolute phi	OT2010 break	Pre-2010 CSI slope	0.000	0.002	-0.004 to 0.003	0.821
Mean absolute phi	OT2010 break	Post-2010 CSI slope	0.029	0.005	0.019 to 0.039	<0.001
Mean absolute phi	OT2010 break	Change in CSI slope	0.030	0.005	0.019 to 0.040	<0.001
Margin- normalized phi	OT2010 break	Pre-2010 CSI slope	0.003	0.003	-0.003 to 0.009	0.335
Margin- normalized phi	OT2010 break	Post-2010 CSI slope	0.022	0.004	0.014 to 0.030	<0.001
Margin- normalized phi	OT2010 break	Change in CSI slope	0.019	0.005	0.009 to 0.029	<0.001

Inferential Robustness Checks

Purpose

The following code evaluates whether the principal inferential result depends on alternative modeling choices or sample definitions.

The robustness analysis first adds SCDB issue-area controls to determine whether the changing salience–alignment relationship can be explained by changes in the substantive composition of the Court’s docket. It then estimates the OT2010 interaction using alternative definitions of a contested case: all non-unanimous cases, the principal two-or-more-minority-vote sample, and cases with at least three minority votes.

A further sensitivity model replaces OT2010 with the **data-driven breakpoint identified by the raw-phi breakpoint analysis**. Because this breakpoint was selected from the same data, its coefficient is treated as a sensitivity estimate rather than as an independent confirmatory test.

Fractional-logit models provide a nonlinear robustness check appropriate to outcomes bounded between zero and one. Finally, a linear-probability model applies the same continuous-salience specification to the intuitive binary strict appointing-party split outcome.

Created objects

- **Robustness_Issue_Results** — OT2010 interaction estimates with issue-area controls.
- **Robustness_Sample_Results** — interaction estimates under alternative contested-case definitions.
- **Robustness_Break_Results** — sensitivity estimates using the data-driven breakpoint.
- **Robustness_Fractional_Results** — fractional-logit interaction estimates.
- **Robustness_Strict_Results** — strict-split linear-probability results.
- **Robustness_Model_Table** — publication-facing comparison of robustness estimates.

```
# --- Inferential robustness checks -----  
  
# -----  
# Helper for extracting one clustered model coefficient  
# -----  
  
extract_clustered_coefficient <- function(  
  model, cluster, coefficient, outcome, specification) {
```

```

vcov_matrix <- cluster_vcov(model, cluster)

cluster_degrees_freedom <- dplyr::n_distinct(cluster) - 1L

cluster_contrast(
  model = model, vcov_matrix = vcov_matrix, weights =
    stats::setNames(1, coefficient), cluster_df =
    cluster_degrees_freedom, outcome = outcome, specification =
    specification, quantity = "Change in CSI slope")
}

# -----
# Issue-area controls
# -----

Model_Data_Issue <- Model_Data %>% dplyr::filter(!is.na(issueArea))

m_phi_post2010_issue <- stats::lm(
  phi_abs ~ csi + csi:Post2010 + factor(term) + factor(issueArea), data =
  Model_Data_Issue)

m_phi_normalized_post2010_issue <- stats::lm(
  phi_normalized ~ csi + csi:Post2010 + factor(term) + factor(issueArea),
  data = Model_Data_Issue)

Robustness_Issue_Results <- dplyr::bind_rows(

  extract_clustered_coefficient(
    model = m_phi_post2010_issue, cluster = Model_Data_Issue$term,
    coefficient = "csi:Post2010", outcome = "Mean absolute phi",
    specification = "OT2010 break + issue-area controls"),

  extract_clustered_coefficient(
    model = m_phi_normalized_post2010_issue, cluster =
    Model_Data_Issue$term, coefficient = "csi:Post2010", outcome =
    "Margin-normalized phi", specification =
    "OT2010 break + issue-area controls"))

# -----
# Alternative definitions of a contested case
# -----

```

```

robustness_samples <- list(

  `All non-unanimous cases` = SCOTUS_NonUnanimous,

  `2+ minority votes` = SCOTUS_Contested,

  `3+ minority votes` = SCOTUS_CloselyDivided)

fit_sample_robustness <- function(
  data, sample_label, outcome) {

  d <- data %>% dplyr::filter(
    term >= analysis_start_term, term <= analysis_end_term, !is.na(csi),
    !is.na(.data[[outcome]])) %>%
    dplyr::mutate(Post2010 = as.integer(term >= 2010))

  model <- stats::lm(
    stats::as.formula(
      paste0(outcome, " ~ csi + csi:Post2010 + factor(term)")), data = d
  )

  extract_clustered_coefficient(
    model = model, cluster = d$term, coefficient = "csi:Post2010", outcome =
      dplyr::case_when(
        outcome == "phi_abs" ~ "Mean absolute phi",
        outcome == "phi_normalized" ~ "Margin-normalized phi", TRUE ~
          outcome), specification =
        paste0("OT2010 break: ", sample_label))
  }

Robustness_Sample_Results <- purrr::imap_dfr(
  robustness_samples, function(data, sample_label) {

    dplyr::bind_rows(

      fit_sample_robustness(
        data = data, sample_label = sample_label, outcome = "phi_abs"),

      fit_sample_robustness(
        data = data, sample_label = sample_label, outcome = "phi_normalized"
      ))
  })
}

```

```

# -----
# Data-driven breakpoint sensitivity
# -----

break_term_sensitivity <- Breakpoint_Best_Raw$Break_Term[
  1]

Model_Data_Break <- Model_Data %>%
  dplyr::mutate(PostBreak = as.integer(term >= break_term_sensitivity))

m_phi_databreak <- stats::lm(
  phi_abs ~ csi + csi:PostBreak + factor(term), data = Model_Data_Break)

m_phi_normalized_databreak <- stats::lm(
  phi_normalized ~ csi + csi:PostBreak + factor(term), data =
    Model_Data_Break)

Robustness_Break_Results <- dplyr::bind_rows(

  extract_clustered_coefficient(
    model = m_phi_databreak, cluster = Model_Data_Break$term, coefficient =
      "csi:PostBreak", outcome = "Mean absolute phi", specification =
        paste0("Data-driven break at OT", break_term_sensitivity)),

  extract_clustered_coefficient(
    model = m_phi_normalized_databreak, cluster = Model_Data_Break$term,
    coefficient = "csi:PostBreak", outcome = "Margin-normalized phi",
    specification =
      paste0("Data-driven break at OT", break_term_sensitivity)))

# -----
# Fractional-logit robustness
# -----

m_phi_fractional <- stats::glm(
  phi_abs ~ csi + csi:Post2010 + factor(term), family =
    stats::quasibinomial(link = "logit"), data = Model_Data)

m_phi_normalized_fractional <- stats::glm(
  phi_normalized ~ csi + csi:Post2010 + factor(term), family =
    stats::quasibinomial(link = "logit"), data = Model_Data)

```

```

Robustness_Fractional_Results <- dplyr::bind_rows(

  extract_clustered_coefficient(
    model = m_phi_fractional, cluster = Model_Data$term, coefficient =
      "csi:Post2010", outcome = "Mean absolute phi", specification =
        "Fractional logit, OT2010 break"),

  extract_clustered_coefficient(
    model = m_phi_normalized_fractional, cluster = Model_Data$term,
    coefficient = "csi:Post2010", outcome = "Margin-normalized phi",
    specification = "Fractional logit, OT2010 break"))

# -----
# Strict appointing-party split as a secondary binary outcome
# -----

Model_Data_Strict <- SCOTUS_Contested %>% dplyr::filter(
  term >= analysis_start_term, term <= analysis_end_term, !is.na(csi),
  !is.na(strict_party_split)) %>% dplyr::mutate(

  Post2010 = as.integer(term >= 2010),

  Strict_Split = as.integer(strict_party_split))

m_strict_lpm <- stats::lm(
  Strict_Split ~ csi + csi:Post2010 + factor(term), data = Model_Data_Strict
)

Robustness_Strict_Results <- extract_clustered_coefficient(
  model = m_strict_lpm, cluster = Model_Data_Strict$term, coefficient =
    "csi:Post2010", outcome = "Strict appointing-party split",
    specification = "Linear probability model, OT2010 break")

# -----
# Combine results
# -----

Robustness_Model_Results <- dplyr::bind_rows(

  Robustness_Issue_Results,

  Robustness_Sample_Results,

```

```

Robustness_Break_Results,

Robustness_Fractional_Results,

Robustness_Strict_Results)

# -----
# Publication-facing robustness table
# -----

Robustness_Model_Table <- Robustness_Model_Results %>% dplyr::mutate(

  `95% CI` = paste0(
    sprintf("%.3f", CI_Lower), " to ", sprintf("%.3f", CI_Upper)),

  `p value` = dplyr::if_else(
    P_Value < 0.001, "<0.001", sprintf("%.3f", P_Value))) %>%
dplyr::transmute(

  Outcome,

  Specification,

  Estimate = round(Estimate, 3),

  `Clustered SE` = round(Clustered_SE, 3),

  `95% CI`,

  `p value`)

knitr::kable(
  Robustness_Model_Table, caption =
    paste0(
      "Robustness of the changing relationship between case salience ",
      "and appointing-party alignment"
    ),
  booktabs = TRUE, align = c("l", "l", "r", "r", "l", "r"))

```

Table 15: Robustness of the changing relationship between case salience and appointing-party alignment

Outcome	Specification	Estimate	Clustered SE	95% CI	p value
Mean absolute phi	OT2010 break + issue-area controls	0.029	0.005	0.018 to 0.040	<0.001
Margin-normalized phi	OT2010 break + issue-area controls	0.018	0.005	0.007 to 0.028	0.001
Mean absolute phi	OT2010 break: All non-unanimous cases	0.032	0.005	0.021 to 0.043	<0.001
Margin-normalized phi	OT2010 break: All non-unanimous cases	0.017	0.005	0.008 to 0.027	<0.001
Mean absolute phi	OT2010 break: 2+ minority votes	0.030	0.005	0.019 to 0.040	<0.001
Margin-normalized phi	OT2010 break: 2+ minority votes	0.019	0.005	0.009 to 0.029	<0.001
Mean absolute phi	OT2010 break: 3+ minority votes	0.029	0.006	0.017 to 0.041	<0.001
Margin-normalized phi	OT2010 break: 3+ minority votes	0.021	0.006	0.009 to 0.034	0.001
Mean absolute phi	Data-driven break at OT2005	0.031	0.004	0.023 to 0.040	<0.001
Margin-normalized phi	Data-driven break at OT2005	0.030	0.006	0.019 to 0.041	<0.001
Mean absolute phi	Fractional logit, OT2010 break	0.144	0.028	0.089 to 0.199	<0.001
Margin-normalized phi	Fractional logit, OT2010 break	0.125	0.031	0.063 to 0.187	<0.001
Strict appointing-party split	Linear probability model, OT2010 break	0.045	0.012	0.021 to 0.069	<0.001